

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan[†]	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	

OpenAI

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks, as well as several tasks that require on-the-fly reasoning or domain adaptation, such as unscrambling words, using a novel word in a sentence, or performing 3-digit arithmetic. At the same time, we also identify some datasets where GPT-3’s few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora. Finally, we find that GPT-3 can generate samples of news articles which human evaluators have difficulty distinguishing from articles written by humans. We discuss broader societal impacts of this finding and of GPT-3 in general.

*Equal contribution

[†]Johns Hopkins University, OpenAI

Contents

1	Introduction	3
2	Approach	6
2.1	Model and Architectures	8
2.2	Training Dataset	8
2.3	Training Process	9
2.4	Evaluation	10
3	Results	10
3.1	Language Modeling, Cloze, and Completion Tasks	11
3.2	Closed Book Question Answering	13
3.3	Translation	14
3.4	Winograd-Style Tasks	16
3.5	Common Sense Reasoning	17
3.6	Reading Comprehension	18
3.7	SuperGLUE	18
3.8	NLI	20
3.9	Synthetic and Qualitative Tasks	21
4	Measuring and Preventing Memorization Of Benchmarks	29
5	Limitations	33
6	Broader Impacts	34
6.1	Misuse of Language Models	35
6.2	Fairness, Bias, and Representation	36
6.3	Energy Usage	39
7	Related Work	39
8	Conclusion	40
A	Details of Common Crawl Filtering	43
B	Details of Model Training	43
C	Details of Test Set Contamination Studies	43
D	Total Compute Used to Train Language Models	46
E	Human Quality Assessment of Synthetic News Articles	46
F	Additional Samples from GPT-3	48
G	Details of Task Phrasing and Specifications	50
H	Results on All Tasks for All Model Sizes	63

1 Introduction

Recent years have featured a trend towards pre-trained language representations in NLP systems, applied in increasingly flexible and task-agnostic ways for downstream transfer. First, single-layer representations were learned using word vectors [MCCD13, PSM14] and fed to task-specific architectures, then RNNs with multiple layers of representations and contextual state were used to form stronger representations [DL15, MBXS17, PNZtY18] (though still applied to task-specific architectures), and more recently pre-trained recurrent or transformer language models [VSP⁺17] have been directly fine-tuned, entirely removing the need for task-specific architectures [RNSS18, DCLT18, HR18].

This last paradigm has led to substantial progress on many challenging NLP tasks such as reading comprehension, question answering, textual entailment, and many others, and has continued to advance based on new architectures and algorithms [RSR⁺19, LOG⁺19, YDY⁺19, LCG⁺19]. However, a major limitation to this approach is that while the architecture is task-agnostic, there is still a need for task-specific datasets and task-specific fine-tuning: to achieve strong performance on a desired task typically requires fine-tuning on a dataset of thousands to hundreds of thousands of examples specific to that task. Removing this limitation would be desirable, for several reasons.

First, from a practical perspective, the need for a large dataset of labeled examples for every new task limits the applicability of language models. There exists a very wide range of possible useful language tasks, encompassing anything from correcting grammar, to generating examples of an abstract concept, to critiquing a short story. For many of these tasks it is difficult to collect a large supervised training dataset, especially when the process must be repeated for every new task.

Second, the potential to exploit spurious correlations in training data fundamentally grows with the expressiveness of the model and the narrowness of the training distribution. This can create problems for the pre-training plus fine-tuning paradigm, where models are designed to be large to absorb information during pre-training, but are then fine-tuned on very narrow task distributions. For instance [HLW⁺20] observe that larger models do not necessarily generalize better out-of-distribution. There is evidence that suggests that the generalization achieved under this paradigm can be poor because the model is overly specific to the training distribution and does not generalize well outside it [YdC⁺19, MPL19]. Thus, the performance of fine-tuned models on specific benchmarks, even when it is nominally at human-level, may exaggerate actual performance on the underlying task [GSL⁺18, NK19].

Third, humans do not require large supervised datasets to learn most language tasks – a brief directive in natural language (e.g. “please tell me if this sentence describes something happy or something sad”) or at most a tiny number of demonstrations (e.g. “here are two examples of people acting brave; please give a third example of bravery”) is often

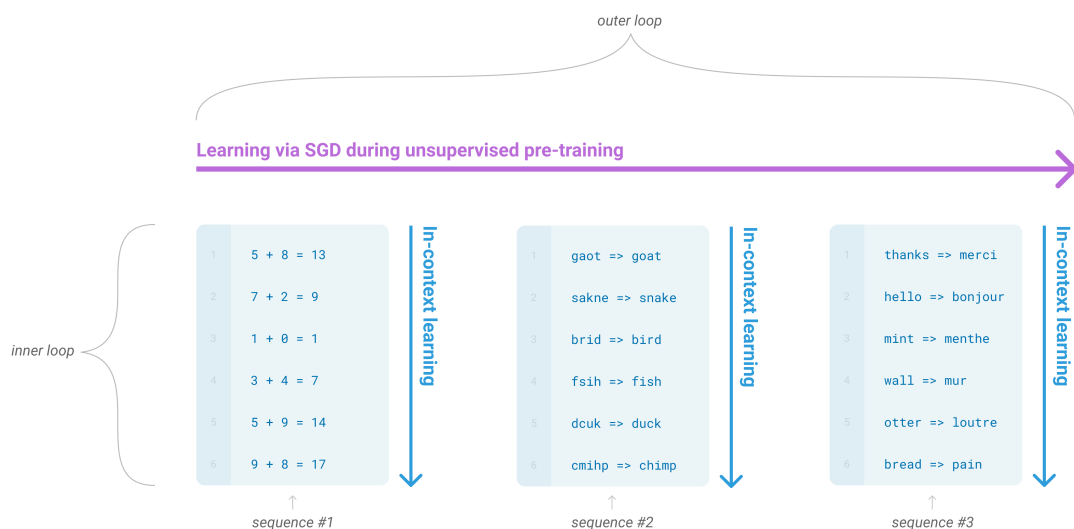


Figure 1.1: Language model meta-learning. During unsupervised pre-training, a language model develops a broad set of skills and pattern recognition abilities. It then uses these abilities at inference time to rapidly adapt to or recognize the desired task. We use the term “in-context learning” to describe the inner loop of this process, which occurs within the forward-pass upon each sequence. The sequences in this diagram are not intended to be representative of the data a model would see during pre-training, but are intended to show that there are sometimes repeated sub-tasks embedded within a single sequence.

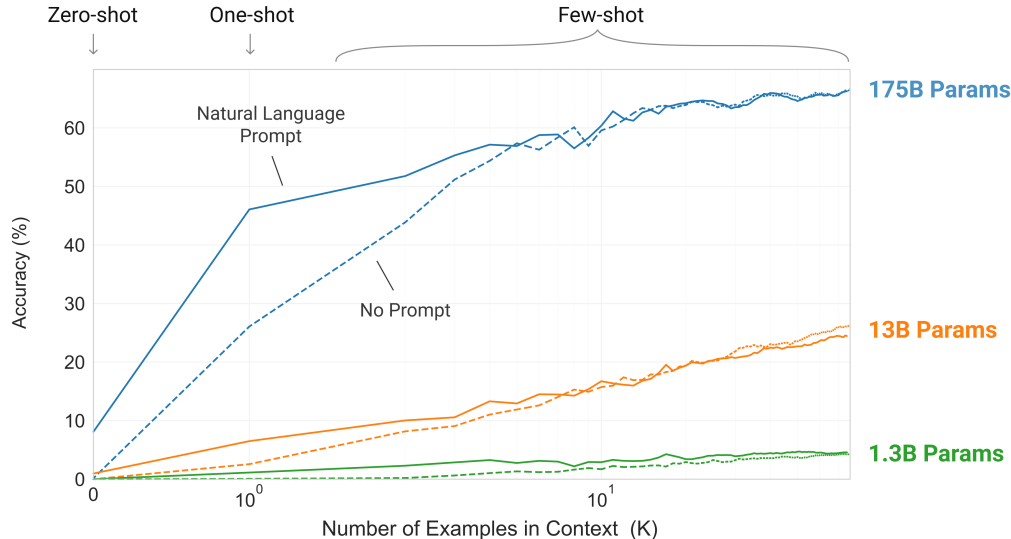


Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

sufficient to enable a human to perform a new task to at least a reasonable degree of competence. Aside from pointing to a conceptual limitation in our current NLP techniques, this adaptability has practical advantages – it allows humans to seamlessly mix together or switch between many tasks and skills, for example performing addition during a lengthy dialogue. To be broadly useful, we would someday like our NLP systems to have this same fluidity and generality.

One potential route towards addressing these issues is meta-learning¹ – which in the context of language models means the model develops a broad set of skills and pattern recognition abilities at training time, and then uses those abilities at inference time to rapidly adapt to or recognize the desired task (illustrated in Figure 1.1). Recent work [RWC⁺19] attempts to do this via what we call “in-context learning”, using the text input of a pretrained language model as a form of task specification: the model is conditioned on a natural language instruction and/or a few demonstrations of the task and is then expected to complete further instances of the task simply by predicting what comes next.

While it has shown some initial promise, this approach still achieves results far inferior to fine-tuning – for example [RWC⁺19] achieves only 4% on Natural Questions, and even its 55 F1 CoQa result is now more than 35 points behind the state of the art. Meta-learning clearly requires substantial improvement in order to be viable as a practical method of solving language tasks.

Another recent trend in language modeling may offer a way forward. In recent years the capacity of transformer language models has increased substantially, from 100 million parameters [RNSS18], to 300 million parameters [DCLT18], to 1.5 billion parameters [RWC⁺19], to 8 billion parameters [SPP⁺19], 11 billion parameters [RSR⁺19], and finally 17 billion parameters [Tur20]. Each increase has brought improvements in text synthesis and/or downstream NLP tasks, and there is evidence suggesting that log loss, which correlates well with many downstream tasks, follows a smooth trend of improvement with scale [KMH⁺20]. Since in-context learning involves absorbing many skills and tasks within the parameters of the model, it is plausible that in-context learning abilities might show similarly strong gains with scale.

¹In the context of language models this has sometimes been called “zero-shot transfer”, but this term is potentially ambiguous: the method is “zero-shot” in the sense that no gradient updates are performed, but it often involves providing inference-time demonstrations to the model, so is not truly learning from zero examples. To avoid this confusion, we use the term “meta-learning” to capture the inner-loop / outer-loop structure of the general method, and the term “in context-learning” to refer to the inner loop of meta-learning. We further specialize the description to “zero-shot”, “one-shot”, or “few-shot” depending on how many demonstrations are provided at inference time. These terms are intended to remain agnostic on the question of whether the model learns new tasks from scratch at inference time or simply recognizes patterns seen during training – this is an important issue which we discuss later in the paper, but “meta-learning” is intended to encompass both possibilities, and simply describes the inner-outer loop structure.

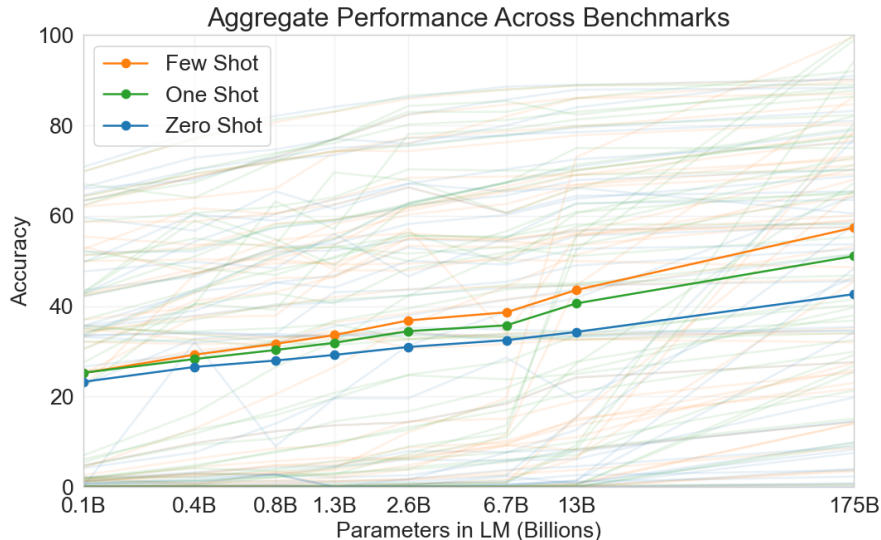


Figure 1.3: Aggregate performance for all 42 accuracy-denominated benchmarks While zero-shot performance improves steadily with model size, few-shot performance increases more rapidly, demonstrating that larger models are more proficient at in-context learning. See Figure 3.8 for a more detailed analysis on SuperGLUE, a standard NLP benchmark suite.

In this paper, we test this hypothesis by training a 175 billion parameter autoregressive language model, which we call GPT-3, and measuring its in-context learning abilities. Specifically, we evaluate GPT-3 on over two dozen NLP datasets, as well as several novel tasks designed to test rapid adaptation to tasks unlikely to be directly contained in the training set. For each task, we evaluate GPT-3 under 3 conditions: (a) “few-shot learning”, or in-context learning where we allow as many demonstrations as will fit into the model’s context window (typically 10 to 100), (b) “one-shot learning”, where we allow only one demonstration, and (c) “zero-shot” learning, where no demonstrations are allowed and only an instruction in natural language is given to the model. GPT-3 could also in principle be evaluated in the traditional fine-tuning setting, but we leave this to future work.

Figure 1.2 illustrates the conditions we study, and shows few-shot learning of a simple task requiring the model to remove extraneous symbols from a word. Model performance improves with the addition of a natural language task description, and with the number of examples in the model’s context, K . Few-shot learning also improves dramatically with model size. Though the results in this case are particularly striking, the general trends with both model size and number of examples in-context hold for most tasks we study. We emphasize that these “learning” curves involve no gradient updates or fine-tuning, just increasing numbers of demonstrations given as conditioning.

Broadly, on NLP tasks GPT-3 achieves promising results in the zero-shot and one-shot settings, and in the the few-shot setting is sometimes competitive with or even occasionally surpasses state-of-the-art (despite state-of-the-art being held by fine-tuned models). For example, GPT-3 achieves 81.5 F1 on CoQA in the zero-shot setting, 84.0 F1 on CoQA in the one-shot setting, 85.0 F1 in the few-shot setting. Similarly, GPT-3 achieves 64.3% accuracy on TriviaQA in the zero-shot setting, 68.0% in the one-shot setting, and 71.2% in the few-shot setting, the last of which is state-of-the-art relative to fine-tuned models operating in the same closed-book setting.

GPT-3 also displays one-shot and few-shot proficiency at tasks designed to test rapid adaption or on-the-fly reasoning, which include unscrambling words, performing arithmetic, and using novel words in a sentence after seeing them defined only once. We also show that in the few-shot setting, GPT-3 can generate synthetic news articles which human evaluators have difficulty distinguishing from human-generated articles.

At the same time, we also find some tasks on which few-shot performance struggles, even at the scale of GPT-3. This includes natural language inference tasks like the ANLI dataset, and some reading comprehension datasets like RACE or QuAC. By presenting a broad characterization of GPT-3’s strengths and weaknesses, including these limitations, we hope to stimulate study of few-shot learning in language models and draw attention to where progress is most needed.

A heuristic sense of the overall results can be seen in Figure 1.3, which aggregates the various tasks (though it should not be seen as a rigorous or meaningful benchmark in itself).

We also undertake a systematic study of “data contamination” – a growing problem when training high capacity models on datasets such as Common Crawl, which can potentially include content from test datasets simply because such content often exists on the web. In this paper we develop systematic tools to measure data contamination and quantify its distorting effects. Although we find that data contamination has a minimal effect on GPT-3’s performance on most datasets, we do identify a few datasets where it could be inflating results, and we either do not report results on these datasets or we note them with an asterisk, depending on the severity.

In addition to all the above, we also train a series of smaller models (ranging from 125 million parameters to 13 billion parameters) in order to compare their performance to GPT-3 in the zero, one and few-shot settings. Broadly, for most tasks we find relatively smooth scaling with model capacity in all three settings; one notable pattern is that the gap between zero-, one-, and few-shot performance often grows with model capacity, perhaps suggesting that larger models are more proficient meta-learners.

Finally, given the broad spectrum of capabilities displayed by GPT-3, we discuss concerns about bias, fairness, and broader societal impacts, and attempt a preliminary analysis of GPT-3’s characteristics in this regard.

The remainder of this paper is organized as follows. In Section 2, we describe our approach and methods for training GPT-3 and evaluating it. Section 3 presents results on the full range of tasks in the zero-, one- and few-shot settings. Section 4 addresses questions of data contamination (train-test overlap). Section 5 discusses limitations of GPT-3. Section 6 discusses broader impacts. Section 7 reviews related work and Section 8 concludes.

2 Approach

Our basic pre-training approach, including model, data, and training, is similar to the process described in [RWC⁺19], with relatively straightforward scaling up of the model size, dataset size and diversity, and length of training. Our use of in-context learning is also similar to [RWC⁺19], but in this work we systematically explore different settings for learning within the context. Therefore, we start this section by explicitly defining and contrasting the different settings that we will be evaluating GPT-3 on or could in principle evaluate GPT-3 on. These settings can be seen as lying on a spectrum of how much task-specific data they tend to rely on. Specifically, we can identify at least four points on this spectrum (see Figure 2.1 for an illustration):

- **Fine-Tuning (FT)** has been the most common approach in recent years, and involves updating the weights of a pre-trained model by training on a supervised dataset specific to the desired task. Typically thousands to hundreds of thousands of labeled examples are used. The main advantage of fine-tuning is strong performance on many benchmarks. The main disadvantages are the need for a new large dataset for every task, the potential for poor generalization out-of-distribution [MPL19], and the potential to exploit spurious features of the training data [GSL⁺18, NK19], potentially resulting in an unfair comparison with human performance. In this work we do not fine-tune GPT-3 because our focus is on task-agnostic performance, but GPT-3 can be fine-tuned in principle and this is a promising direction for future work.
- **Few-Shot (FS)** is the term we will use in this work to refer to the setting where the model is given a few demonstrations of the task at inference time as conditioning [RWC⁺19], but no weight updates are allowed. As shown in Figure 2.1, for a typical dataset an example has a context and a desired completion (for example an English sentence and the French translation), and few-shot works by giving K examples of context and completion, and then one final example of context, with the model expected to provide the completion. We typically set K in the range of 10 to 100 as this is how many examples can fit in the model’s context window ($n_{\text{ctx}} = 2048$). The main advantages of few-shot are a major reduction in the need for task-specific data and reduced potential to learn an overly narrow distribution from a large but narrow fine-tuning dataset. The main disadvantage is that results from this method have so far been much worse than state-of-the-art fine-tuned models. Also, a small amount of task specific data is still required. As indicated by the name, few-shot learning as described here for language models is related to few-shot learning as used in other contexts in ML [HYC01, VBL⁺16] – both involve learning based on a broad distribution of tasks (in this case implicit in the pre-training data) and then rapidly adapting to a new task.
- **One-Shot (1S)** is the same as few-shot except that only one demonstration is allowed, in addition to a natural language description of the task, as shown in Figure 1. The reason to distinguish one-shot from few-shot and zero-shot (below) is that it most closely matches the way in which some tasks are communicated to humans. For example, when asking humans to generate a dataset on a human worker service (for example Mechanical Turk), it is common to give one demonstration of the task. By contrast it is sometimes difficult to communicate the content or format of a task if no examples are given.

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ← examples
4 plush giraffe => girafe peluche ← examples
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Figure 2.1: Zero-shot, one-shot and few-shot, contrasted with traditional fine-tuning. The panels above show four methods for performing a task with a language model – fine-tuning is the traditional method, whereas zero-, one-, and few-shot, which we study in this work, require the model to perform the task with only forward passes at test time. We typically present the model with a few dozen examples in the few shot setting. Exact phrasings for all task descriptions, examples and prompts can be found in Appendix G.

- **Zero-Shot (0S)** is the same as one-shot except that no demonstrations are allowed, and the model is only given a natural language instruction describing the task. This method provides maximum convenience, potential for robustness, and avoidance of spurious correlations (unless they occur very broadly across the large corpus of pre-training data), but is also the most challenging setting. In some cases it may even be difficult for humans to understand the format of the task without prior examples, so this setting is in some cases “unfairly hard”. For example, if someone is asked to “make a table of world records for the 200m dash”, this request can be ambiguous, as it may not be clear exactly what format the table should have or what should be included (and even with careful clarification, understanding precisely what is desired can be difficult). Nevertheless, for at least some settings zero-shot is closest to how humans perform tasks – for example, in the translation example in Figure 2.1, a human would likely know what to do from just the text instruction.

Figure 2.1 shows the four methods using the example of translating English to French. In this paper we focus on zero-shot, one-shot and few-shot, with the aim of comparing them not as competing alternatives, but as different problem settings which offer a varying trade-off between performance on specific benchmarks and sample efficiency. We especially highlight the few-shot results as many of them are only slightly behind state-of-the-art fine-tuned models. Ultimately, however, one-shot, or even sometimes zero-shot, seem like the fairest comparisons to human performance, and are important targets for future work.

Sections 2.1-2.3 below give details on our models, training data, and training process respectively. Section 2.4 discusses the details of how we do few-shot, one-shot, and zero-shot evaluations.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

2.1 Model and Architectures

We use the same model and architecture as GPT-2 [RWC⁺19], including the modified initialization, pre-normalization, and reversible tokenization described therein, with the exception that we use alternating dense and locally banded sparse attention patterns in the layers of the transformer, similar to the Sparse Transformer [CGRS19]. To study the dependence of ML performance on model size, we train 8 different sizes of model, ranging over three orders of magnitude from 125 million parameters to 175 billion parameters, with the last being the model we call GPT-3. Previous work [KMH⁺20] suggests that with enough training data, scaling of validation loss should be approximately a smooth power law as a function of size; training models of many different sizes allows us to test this hypothesis both for validation loss and for downstream language tasks.

Table 2.1 shows the sizes and architectures of our 8 models. Here n_{params} is the total number of trainable parameters, n_{layers} is the total number of layers, d_{model} is the number of units in each bottleneck layer (we always have the feedforward layer four times the size of the bottleneck layer, $d_{\text{ff}} = 4 * d_{\text{model}}$), and d_{head} is the dimension of each attention head. All models use a context window of $n_{\text{ctx}} = 2048$ tokens. We partition the model across GPUs along both the depth and width dimension in order to minimize data-transfer between nodes. The precise architectural parameters for each model are chosen based on computational efficiency and load-balancing in the layout of models across GPU’s. Previous work [KMH⁺20] suggests that validation loss is not strongly sensitive to these parameters within a reasonably broad range.

2.2 Training Dataset

Datasets for language models have rapidly expanded, culminating in the Common Crawl dataset² [RSR⁺19] constituting nearly a trillion words. This size of dataset is sufficient to train our largest models without ever updating on the same sequence twice. However, we have found that unfiltered or lightly filtered versions of Common Crawl tend to have lower quality than more curated datasets. Therefore, we took 3 steps to improve the average quality of our datasets: (1) we downloaded and filtered a version of CommonCrawl based on similarity to a range of high-quality reference corpora, (2) we performed fuzzy deduplication at the document level, within and across datasets, to prevent redundancy and preserve the integrity of our held-out validation set as an accurate measure of overfitting, and (3) we also added known high-quality reference corpora to the training mix to augment CommonCrawl and increase its diversity.

Details of the first two points (processing of Common Crawl) are described in Appendix A. For the third, we added several curated high-quality datasets, including an expanded version of the WebText dataset [RWC⁺19], collected by scraping links over a longer period of time, and first described in [KMH⁺20], two internet-based books corpora (Books1 and Books2) and English-language Wikipedia.

Table 2.2 shows the final mixture of datasets that we used in training. The CommonCrawl data was downloaded from 41 shards of monthly CommonCrawl covering 2016 to 2019, constituting 45TB of compressed plaintext before filtering and 570GB after filtering, roughly equivalent to 400 billion byte-pair-encoded tokens. Note that during training, datasets are not sampled in proportion to their size, but rather datasets we view as higher-quality are sampled more frequently, such that CommonCrawl and Books2 datasets are sampled less than once during training, but the other datasets are sampled 2-3 times. This essentially accepts a small amount of overfitting in exchange for higher quality training data.

²<https://commoncrawl.org/the-data/>

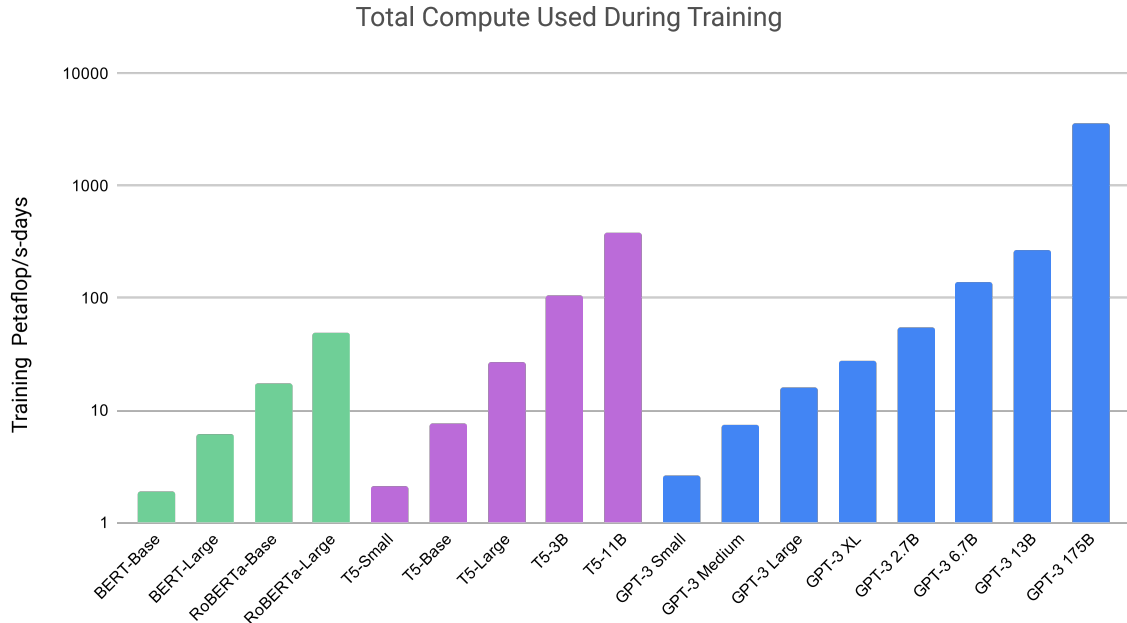


Figure 2.2: Total compute used during training. Based on the analysis in Scaling Laws For Neural Language Models [KMH⁺20] we train much larger models on many fewer tokens than is typical. As a consequence, although GPT-3 3B is almost 10x larger than RoBERTa-Large (355M params), both models took roughly 50 petaflop/s-days of compute during pre-training. Methodology for these calculations can be found in Appendix D.

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

A major methodological concern with language models pretrained on a broad swath of internet data, particularly large models with the capacity to memorize vast amounts of content, is potential contamination of downstream tasks by having their test or development sets inadvertently seen during pre-training. To reduce such contamination, we searched for and attempted to remove any overlaps with the development and test sets of all benchmarks studied in this paper. Unfortunately, a bug in the filtering caused us to ignore some overlaps, and due to the cost of training it was not feasible to retrain the model. In Section 4 we characterize the impact of the remaining overlaps, and in future work we will more aggressively remove data contamination.

2.3 Training Process

As found in [KMH⁺20, MKAT18], larger models can typically use a larger batch size, but require a smaller learning rate. We measure the gradient noise scale during training and use it to guide our choice of batch size [MKAT18]. Table 2.1 shows the parameter settings we used. To train the larger models without running out of memory, we use a mixture of model parallelism within each matrix multiply and model parallelism across the layers of the network. All models were trained on V100 GPU’s on part of a high-bandwidth cluster provided by Microsoft. Details of the training process and hyperparameter settings are described in Appendix B.

2.4 Evaluation

For few-shot learning, we evaluate each example in the evaluation set by randomly drawing K examples from that task’s training set as conditioning, delimited by 1 or 2 newlines depending on the task. For LAMBADA and Storycloze there is no supervised training set available so we draw conditioning examples from the development set and evaluate on the test set. For Winograd (the original, not SuperGLUE version) there is only one dataset, so we draw conditioning examples directly from it.

K can be any value from 0 to the maximum amount allowed by the model’s context window, which is $n_{\text{ctx}} = 2048$ for all models and typically fits 10 to 100 examples. Larger values of K are usually but not always better, so when a separate development and test set are available, we experiment with a few values of K on the development set and then run the best value on the test set. For some tasks (see Appendix G) we also use a natural language prompt in addition to (or for $K = 0$, instead of) demonstrations.

On tasks that involve choosing one correct completion from several options (multiple choice), we provide K examples of context plus correct completion, followed by one example of context only, and compare the LM likelihood of each completion. For most tasks we compare the per-token likelihood (to normalize for length), however on a small number of datasets (ARC, OpenBookQA, and RACE) we gain additional benefit as measured on the development set by normalizing by the unconditional probability of each completion, by computing $\frac{P(\text{completion}|\text{context})}{P(\text{completion}|\text{answer_context})}$, where `answer_context` is the string "Answer: " or "A: " and is used to prompt that the completion should be an answer but is otherwise generic.

On tasks that involve binary classification, we give the options more semantically meaningful names (e.g. “True” or “False” rather than 0 or 1) and then treat the task like multiple choice; we also sometimes frame the task similar to what is done by [RSR⁺19] (see Appendix G) for details.

On tasks with free-form completion, we use beam search with the same parameters as [RSR⁺19]: a beam width of 4 and a length penalty of $\alpha = 0.6$. We score the model using F1 similarity score, BLEU, or exact match, depending on what is standard for the dataset at hand.

Final results are reported on the test set when publicly available, for each model size and learning setting (zero-, one-, and few-shot). When the test set is private, our model is often too large to fit on the test server, so we report results on the development set. We do submit to the test server on a small number of datasets (SuperGLUE, TriviaQA, PiQA) where we were able to make submission work, and we submit only the 200B few-shot results, and report development set results for everything else.

3 Results

In Figure 3.1 we display training curves for the 8 models described in Section 2. For this graph we also include 6 additional extra-small models with as few as 100,000 parameters. As observed in [KMH⁺20], language modeling performance follows a power-law when making efficient use of training compute. After extending this trend by two more orders of magnitude, we observe only a slight (if any) departure from the power-law. One might worry that these improvements in cross-entropy loss come only from modeling spurious details of our training corpus. However, we will see in the following sections that improvements in cross-entropy loss lead to consistent performance gains across a broad spectrum of natural language tasks.

Below, we evaluate the 8 models described in Section 2 (the 175 billion parameter parameter GPT-3 and 7 smaller models) on a wide range of datasets. We group the datasets into 9 categories representing roughly similar tasks.

In Section 3.1 we evaluate on traditional language modeling tasks and tasks that are similar to language modeling, such as Cloze tasks and sentence/paragraph completion tasks. In Section 3.2 we evaluate on “closed book” question answering tasks: tasks which require using the information stored in the model’s parameters to answer general knowledge questions. In Section 3.3 we evaluate the model’s ability to translate between languages (especially one-shot and few-shot). In Section 3.4 we evaluate the model’s performance on Winograd Schema-like tasks. In Section 3.5 we evaluate on datasets that involve commonsense reasoning or question answering. In Section 3.6 we evaluate on reading comprehension tasks, in Section 3.7 we evaluate on the SuperGLUE benchmark suite, and in 3.8 we briefly explore NLI. Finally, in Section 3.9, we invent some additional tasks designed especially to probe in-context learning abilities – these tasks focus on on-the-fly reasoning, adaptation skills, or open-ended text synthesis. We evaluate all tasks in the few-shot, one-shot, and zero-shot settings.

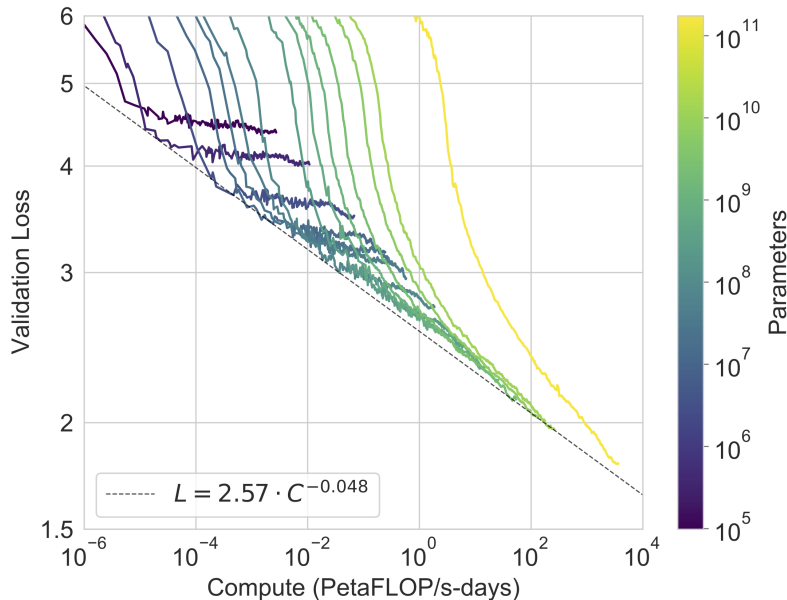


Figure 3.1: Smooth scaling of performance with compute. Performance (measured in terms of cross-entropy validation loss) follows a power-law trend with the amount of compute used for training. The power-law behavior observed in [KMH⁺20] continues for an additional two orders of magnitude with only small deviations from the predicted curve. For this figure, we exclude embedding parameters from compute and parameter counts.

Setting	PTB
SOTA (Zero-Shot)	35.8 ^a
GPT-3 Zero-Shot	20.5

Table 3.1: Zero-shot results on PTB language modeling dataset. Many other common language modeling datasets are omitted because they are derived from Wikipedia or other sources which are included in GPT-3’s training data.
^a[RWC⁺19]

3.1 Language Modeling, Cloze, and Completion Tasks

In this section we test GPT-3’s performance on the traditional task of language modeling, as well as related tasks that involve predicting a single word of interest, completing a sentence or paragraph, or choosing between possible completions of a piece of text.

3.1.1 Language Modeling

We calculate zero-shot perplexity on the Penn Tree Bank (PTB) [MKM⁺94] dataset measured in [RWC⁺19]. We omit the 4 Wikipedia-related tasks in that work because they are entirely contained in our training data, and we also omit the one-billion word benchmark due to a high fraction of the dataset being contained in our training set. PTB escapes these issues due to predating the modern internet. Our largest model sets a new SOTA on PTB by a substantial margin of 15 points, achieving a perplexity of 20.50. Note that since PTB is a traditional language modeling dataset it does not have a clear separation of examples to define one-shot or few-shot evaluation around, so we measure only zero-shot.

3.1.2 LAMBADA

The LAMBADA dataset [PKL⁺16] tests the modeling of long-range dependencies in text – the model is asked to predict the last word of sentences which require reading a paragraph of context. It has recently been suggested that the continued scaling of language models is yielding diminishing returns on this difficult benchmark. [BHT⁺20] reflect on the small 1.5% improvement achieved by a doubling of model size between two recent state of the art results ([SPP⁺19]

Setting	LAMBADA (acc)	LAMBADA (ppl)	StoryCloze (acc)	HellaSwag (acc)
SOTA	68.0 ^a	8.63 ^b	91.8^c	85.6^d
GPT-3 Zero-Shot	76.2	3.00	83.2	78.9
GPT-3 One-Shot	72.5	3.35	84.7	78.1
GPT-3 Few-Shot	86.4	1.92	87.7	79.3

Table 3.2: Performance on cloze and completion tasks. GPT-3 significantly improves SOTA on LAMBADA while achieving respectable performance on two difficult completion prediction datasets. ^a[Tur20] ^b[RWC⁺19] ^c[LDL19] ^d[LCH⁺20]

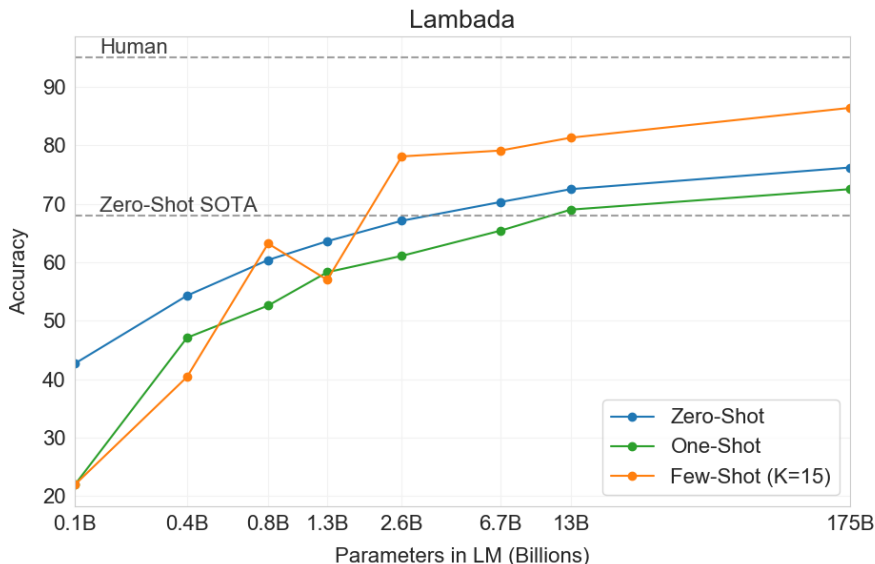


Figure 3.2: On LAMBADA, the few-shot capability of language models results in a strong boost to accuracy. GPT-3 2.7B outperforms the SOTA 17B parameter Turing-NLG [Tur20] in this setting, and GPT-3 175B advances the state of the art by 18%. Note zero-shot uses a different format from one-shot and few-shot as described in the text.

and [Tur20]) and argue that “continuing to expand hardware and data sizes by orders of magnitude is not the path forward”. We find that path is still promising and in a zero-shot setting GPT-3 achieves 76% on LAMBADA, a gain of 8% over the previous state of the art.

LAMBADA is also a demonstration of the flexibility of few-shot learning as it provides a way to address a problem that classically occurs with this dataset. Although the completion in LAMBADA is always the last word in a sentence, a standard language model has no way of knowing this detail. It thus assigns probability not only to the correct ending but also to other valid continuations of the paragraph. This problem has been partially addressed in the past with stop-word filters [RWC⁺19] (which ban “continuation” words). The few-shot setting instead allows us to “frame” the task as a cloze-test and allows the language model to infer from examples that a completion of exactly one word is desired. We use the following fill-in-the-blank format:

Alice was friends with Bob. Alice went to visit her friend _____. → Bob

George bought some baseball equipment, a ball, a glove, and a _____. →

When presented with examples formatted this way, GPT-3 achieves 86.4% accuracy in the few-shot setting, an increase of over 18% from the previous state-of-the-art. We observe that few-shot performance improves strongly with model size. While this setting decreases the performance of the smallest model by almost 20%, for GPT-3 it improves accuracy by 10%. Finally, the fill-in-blank method is not effective one-shot, where it always performs worse than the zero-shot setting. Perhaps this is because all models still require several examples to recognize the pattern.

Setting	NaturalQS	WebQS	TriviaQA
RAG (Fine-tuned, Open-Domain) [LPP ⁺ 20]	44.5	45.5	68.0
T5-11B+SSM (Fine-tuned, Closed-Book) [RRS20]	36.6	44.7	60.5
T5-11B (Fine-tuned, Closed-Book)	34.5	37.4	50.1
GPT-3 Zero-Shot	14.6	14.4	64.3
GPT-3 One-Shot	23.0	25.3	68.0
GPT-3 Few-Shot	29.9	41.5	71.2

Table 3.3: Results on three Open-Domain QA tasks. GPT-3 is shown in the few-, one-, and zero-shot settings, as compared to prior SOTA results for closed book and open domain settings. TriviaQA few-shot result is evaluated on the wiki split test server.

One note of caution is that an analysis of test set contamination identified that a significant minority of the LAMBADA dataset appears to be present in our training data – however analysis performed in Section 4 suggests negligible impact on performance.

3.1.3 HellaSwag

The HellaSwag dataset [ZHB⁺19] involves picking the best ending to a story or set of instructions. The examples were adversarially mined to be difficult for language models while remaining easy for humans (who achieve 95.6% accuracy). GPT-3 achieves 78.1% accuracy in the one-shot setting and 79.3% accuracy in the few-shot setting, outperforming the 75.4% accuracy of a fine-tuned 1.5B parameter language model [ZHR⁺19] but still a fair amount lower than the overall SOTA of 85.6% achieved by the fine-tuned multi-task model ALUM.

3.1.4 StoryCloze

We next evaluate GPT-3 on the StoryCloze 2016 dataset [MCH⁺16], which involves selecting the correct ending sentence for five-sentence long stories. Here GPT-3 achieves 83.2% in the zero-shot setting and 87.7% in the few-shot setting (with $K = 70$). This is still 4.1% lower than the fine-tuned SOTA using a BERT based model [LDL19] but improves over previous zero-shot results by roughly 10%.

3.2 Closed Book Question Answering

In this section we measure GPT-3’s ability to answer questions about broad factual knowledge. Due to the immense amount of possible queries, this task has normally been approached by using an information retrieval system to find relevant text in combination with a model which learns to generate an answer given the question and the retrieved text. Since this setting allows a system to search for and condition on text which potentially contains the answer it is denoted “open-book”. [RRS20] recently demonstrated that a large language model can perform surprisingly well directly answering the questions without conditioning on auxiliary information. They denote this more restrictive evaluation setting as “closed-book”. Their work suggests that even higher-capacity models could perform even better and we test this hypothesis with GPT-3. We evaluate GPT-3 on the 3 datasets in [RRS20]: Natural Questions [KPR⁺19], WebQuestions [BCFL13], and TriviaQA [JCWZ17], using the same splits. Note that in addition to all results being in the closed-book setting, our use of few-shot, one-shot, and zero-shot evaluations represent an even stricter setting than previous closed-book QA work: in addition to external content not being allowed, fine-tuning on the Q&A dataset itself is also not permitted.

The results for GPT-3 are shown in Table 3.3. On TriviaQA, we achieve 64.3% in the zero-shot setting, 68.0% in the one-shot setting, and 71.2% in the few-shot setting. The zero-shot result already outperforms the fine-tuned T5-11B by 14.2%, and also outperforms a version with Q&A tailored span prediction during pre-training by 3.8%. The one-shot result improves by 3.7% and matches the SOTA for an open-domain QA system which not only fine-tunes but also makes use of a learned retrieval mechanism over a 15.3B parameter dense vector index of 21M documents [LPP⁺20]. GPT-3’s few-shot result further improves performance another 3.2% beyond this.

On WebQuestions (WebQs), GPT-3 achieves 14.4% in the zero-shot setting, 25.3% in the one-shot setting, and 41.5% in the few-shot setting. This compares to 37.4% for fine-tuned T5-11B, and 44.7% for fine-tuned T5-11B+SSM, which uses a Q&A-specific pre-training procedure. GPT-3 in the few-shot setting approaches the performance of state-of-the-art fine-tuned models. Notably, compared to TriviaQA, WebQS shows a much larger gain from zero-shot to few-shot (and indeed its zero-shot and one-shot performance are poor), perhaps suggesting that the WebQs questions

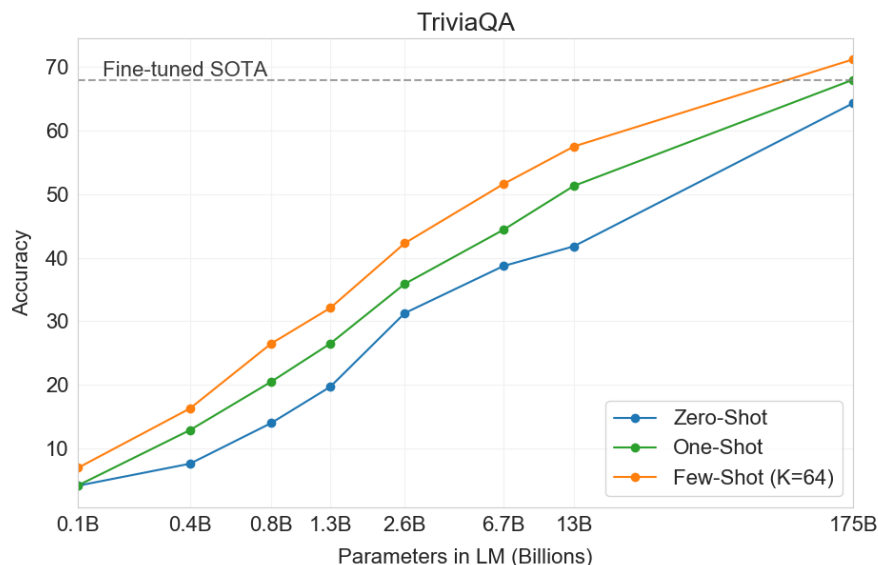


Figure 3.3: On TriviaQA GPT3’s performance grows smoothly with model size, suggesting that language models continue to absorb knowledge as their capacity increases. One-shot and few-shot performance make significant gains over zero-shot behavior, matching and exceeding the performance of the SOTA fine-tuned open-domain model, RAG [LPP⁺20]

and/or the style of their answers are out-of-distribution for GPT-3. Nevertheless, GPT-3 appears able to adapt to this distribution, recovering strong performance in the few-shot setting.

On Natural Questions (NQs) GPT-3 achieves 14.6% in the zero-shot setting, 23.0% in the one-shot setting, and 29.9% in the few-shot setting, compared to 36.6% for fine-tuned T5 11B+SSM. Similar to WebQS, the large gain from zero-shot to few-shot may suggest a distribution shift, and may also explain the less competitive performance compared to TriviaQA and WebQS. In particular, the questions in NQs tend towards very fine-grained knowledge on Wikipedia specifically which could be testing the limits of GPT-3’s capacity and broad pretraining distribution.

Overall, on one of the three datasets GPT-3’s one-shot matches the open-domain fine-tuning SOTA. On the other two datasets it approaches the performance of the closed-book SOTA despite not using fine-tuning. On all 3 datasets, we find that performance scales very smoothly with model size (Figure 3.3 and Appendix H Figure H.7), possibly reflecting the idea that model capacity translates directly to more ‘knowledge’ absorbed in the parameters of the model.

3.3 Translation

For GPT-2 a filter was used on a multilingual collection of documents to produce an English only dataset due to capacity concerns. Even with this filtering GPT-2 showed some evidence of multilingual capability and performed non-trivially when translating between French and English despite only training on 10 megabytes of remaining French text. Since we increase the capacity by over two orders of magnitude from GPT-2 to GPT-3, we also expand the scope of the training dataset to include more representation of other languages, though this remains an area for further improvement. As discussed in 2.2 the majority of our data is derived from raw Common Crawl with only quality-based filtering. Although GPT-3’s training data is still primarily English (93% by word count), it also includes 7% of text in other languages. These languages are documented in the supplemental material. In order to better understand translation capability, we also expand our analysis to include two additional commonly studied languages, German and Romanian.

Existing unsupervised machine translation approaches often combine pretraining on a pair of monolingual datasets with back-translation [SHB15] to bridge the two languages in a controlled way. By contrast, GPT-3 learns from a blend of training data that mixes many languages together in a natural way, combining them on a word, sentence, and document level. GPT-3 also uses a single training objective which is not customized or designed for any task in particular. However, our one / few-shot settings aren’t strictly comparable to prior unsupervised work since they make use of a small amount of paired examples (1 or 64). This corresponds to up to a page or two of in-context training data.

Results are shown in Table 3.4. Zero-shot GPT-3, which only receives on a natural language description of the task, still underperforms recent unsupervised NMT results. However, providing only a single example demonstration for

Setting	En→Fr	Fr→En	En→De	De→En	En→Ro	Ro→En
SOTA (Supervised)	45.6^a	35.0 ^b	41.2^c	40.2 ^d	38.5^e	39.9^e
XLM [LC19]	33.4	33.3	26.4	34.3	33.3	31.8
MASS [STQ ⁺ 19]	<u>37.5</u>	34.9	28.3	35.2	<u>35.2</u>	33.1
mBART [LGG ⁺ 20]	-	-	<u>29.8</u>	34.0	35.0	30.5
GPT-3 Zero-Shot	25.2	21.2	24.6	27.2	14.1	19.9
GPT-3 One-Shot	28.3	33.7	26.2	30.4	20.6	38.6
GPT-3 Few-Shot	32.6	<u>39.2</u>	29.7	<u>40.6</u>	21.0	<u>39.5</u>

Table 3.4: Few-shot GPT-3 outperforms previous unsupervised NMT work by 5 BLEU when translating into English reflecting its strength as an English LM. We report BLEU scores on the WMT’14 Fr↔En, WMT’16 De↔En, and WMT’16 Ro↔En datasets as measured by multi-bleu.perl with XLM’s tokenization in order to compare most closely with prior unsupervised NMT work. SacreBLEU^f [Pos18] results reported in Appendix H. Underline indicates an unsupervised or few-shot SOTA, bold indicates supervised SOTA with relative confidence. ^a[EOAG18] ^b[DHKH14] ^c[WXH⁺18] ^d[oR16] ^e[LGG⁺20] ^f[SacreBLEU signature: BLEU+case.mixed+numrefs.1+smooth.exp+tok.intl+version.1.2.20]

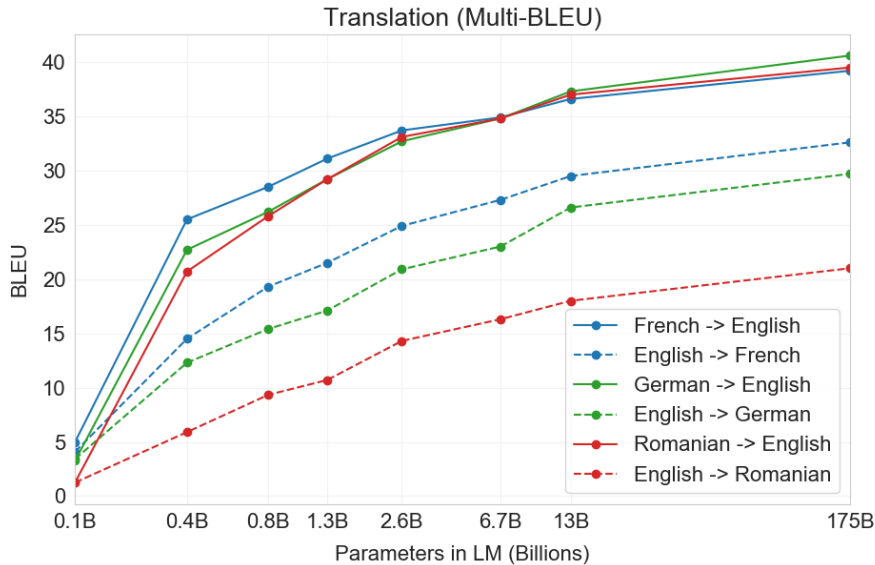


Figure 3.4: Few-shot translation performance on 6 language pairs as model capacity increases. There is a consistent trend of improvement across all datasets as the model scales, and as well as tendency for translation into English to be stronger than translation from English.

Setting	Winograd	Winogrande (XL)
Fine-tuned SOTA	90.1^a	84.6^b
GPT-3 Zero-Shot	88.3*	70.2
GPT-3 One-Shot	89.7*	73.2
GPT-3 Few-Shot	88.6*	77.7

Table 3.5: Results on the WSC273 version of Winograd schemas and the adversarial Winogrande dataset. See Section 4 for details on potential contamination of the Winograd test set. ^a[SBBC19] ^b[LYN+20]



Figure 3.5: Zero-, one-, and few-shot performance on the adversarial Winogrande dataset as model capacity scales. Scaling is relatively smooth with the gains to few-shot learning increasing with model size, and few-shot GPT-3 175B is competitive with a fine-tuned RoBERTa-large.

each translation task improves performance by over 7 BLEU and nears competitive performance with prior work. GPT-3 in the full few-shot setting further improves another 4 BLEU resulting in similar average performance to prior unsupervised NMT work. GPT-3 has a noticeable skew in its performance depending on language direction. For the three input languages studied, GPT-3 significantly outperforms prior unsupervised NMT work when translating into English but underperforms when translating in the other direction. Performance on En-Ro is a noticeable outlier at over 10 BLEU worse than prior unsupervised NMT work. This could be a weakness due to reusing the byte-level BPE tokenizer of GPT-2 which was developed for an almost entirely English training dataset. For both Fr-En and De-En, few shot GPT-3 outperforms the best supervised result we could find but due to our unfamiliarity with the literature and the appearance that these are un-competitive benchmarks we do not suspect those results represent true state of the art. For Ro-En, few shot GPT-3 performs within 0.5 BLEU of the overall SOTA which is achieved by a combination of unsupervised pretraining, supervised finetuning on 608K labeled examples, and backtranslation [LHCG19b].

Finally, across all language pairs and across all three settings (zero-, one-, and few-shot), there is a smooth trend of improvement with model capacity. This is shown in Figure 3.4 in the case of few-shot results, and scaling for all three settings is shown in Appendix H.

3.4 Winograd-Style Tasks

The Winograd Schemas Challenge [LDM12] is a classical task in NLP that involves determining which word a pronoun refers to, when the pronoun is grammatically ambiguous but semantically unambiguous to a human. Recently fine-tuned language models have achieved near-human performance on the original Winograd dataset, but more difficult versions

Setting	PIQA	ARC (Easy)	ARC (Challenge)	OpenBookQA
Fine-tuned SOTA	79.4	92.0 [KKS+20]	78.5 [KKS+20]	87.2 [KKS+20]
GPT-3 Zero-Shot	80.5*	68.8	51.4	57.6
GPT-3 One-Shot	80.5*	71.2	53.2	58.8
GPT-3 Few-Shot	82.8*	70.1	51.5	65.4

Table 3.6: GPT-3 results on three commonsense reasoning tasks, PIQA, ARC, and OpenBookQA. GPT-3 Few-Shot PIQA result is evaluated on the test server. See Section 4 for details on potential contamination issues on the PIQA test set.

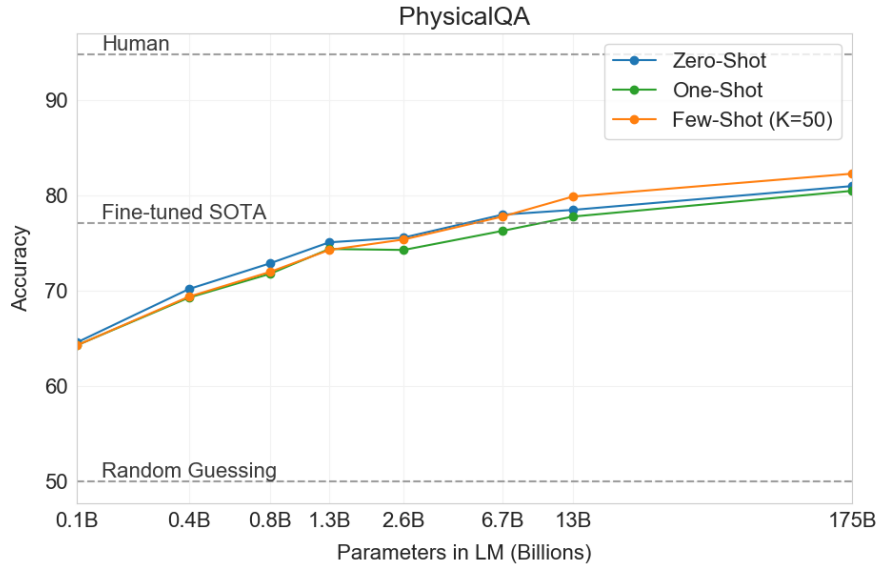


Figure 3.6: GPT-3 results on PIQA in the zero-shot, one-shot, and few-shot settings. The largest model achieves a score on the development set in all three conditions that exceeds the best recorded score on the task.

such as the adversarially-mined Winogrande dataset [SBBC19] still significantly lag human performance. We test GPT-3’s performance on both Winograd and Winogrande, as usual in the zero-, one-, and few-shot setting.

On Winograd we test GPT-3 on the original set of 273 Winograd schemas, using the same “partial evaluation” method described in [RWC+19]. Note that this setting differs slightly from the WSC task in the SuperGLUE benchmark, which is presented as binary classification and requires entity extraction to convert to the form described in this section. On Winograd GPT-3 achieves 88.3%, 89.7%, and 88.6% in the zero-shot, one-shot, and few-shot settings, showing no clear in-context learning but in all cases achieving strong results just a few points below state-of-the-art and estimated human performance. We note that contamination analysis found some Winograd schemas in the training data but this appears to have only a small effect on results (see Section 4).

On the more difficult Winogrande dataset, we do find gains to in-context learning: GPT-3 achieves 70.2% in the zero-shot setting, 73.2% in the one-shot setting, and 77.7% in the few-shot setting. For comparison a fine-tuned RoBERTa model achieves 79%, state-of-the-art is 84.6% achieved with a fine-tuned high capacity model (T5), and human performance on the task as reported by [SBBC19] is 94.0%.

3.5 Common Sense Reasoning

Next we consider three datasets which attempt to capture physical or scientific reasoning, as distinct from sentence completion, reading comprehension, or broad knowledge question answering. The first, PhysicalQA (PIQA) [BZB+19], asks common sense questions about how the physical world works and is intended as a probe of grounded understanding of the world. GPT-3 achieves 81.0% accuracy zero-shot, 80.5% accuracy one-shot, and 82.8% accuracy few-shot (the last measured on PIQA’s test server). This compares favorably to the 79.4% accuracy prior state-of-the-art of a

Setting	CoQA	DROP	QuAC	SQuADv2	RACE-h	RACE-m
Fine-tuned SOTA	90.7^a	89.1^b	74.4^c	93.0^d	90.0^e	93.1^e
GPT-3 Zero-Shot	81.5	23.6	41.5	59.5	45.5	58.4
GPT-3 One-Shot	84.0	34.3	43.3	65.4	45.9	57.4
GPT-3 Few-Shot	85.0	36.5	44.3	69.8	46.8	58.1

Table 3.7: Results on reading comprehension tasks. All scores are F1 except results for RACE which report accuracy. ^a[JZC⁺19] ^b[JN20] ^c[AI19] ^d[QIA20] ^e[SPP⁺19]

fine-tuned RoBERTa. PIQA shows relatively shallow scaling with model size and is still over 10% worse than human performance, but GPT-3’s few-shot and even zero-shot result outperform the current state-of-the-art. Our analysis flagged PIQA for a potential data contamination issue (despite hidden test labels), and we therefore conservatively mark the result with an asterisk. See Section 4 for details.

ARC [CCE⁺18] is a dataset of multiple-choice questions collected from 3rd to 9th grade science exams. On the “Challenge” version of the dataset which has been filtered to questions which simple statistical or information retrieval methods are unable to correctly answer, GPT-3 achieves 51.4% accuracy in the zero-shot setting, 53.2% in the one-shot setting, and 51.5% in the few-shot setting. This is approaching the performance of a fine-tuned RoBERTa baseline (55.9%) from UnifiedQA [KKS⁺20]. On the “Easy” version of the dataset (questions which either of the mentioned baseline approaches answered correctly), GPT-3 achieves 68.8%, 71.2%, and 70.1% which slightly exceeds a fine-tuned RoBERTa baseline from [KKS⁺20]. However, both of these results are still much worse than the overall SOTAs achieved by the UnifiedQA which exceeds GPT-3’s few-shot results by 27% on the challenge set and 22% on the easy set.

On OpenBookQA [MCKS18], GPT-3 improves significantly from zero to few shot settings but is still over 20 points short of the overall SOTA. GPT-3’s few-shot performance is similar to a fine-tuned BERT Large baseline on the leaderboard.

Overall, in-context learning with GPT-3 shows mixed results on commonsense reasoning tasks, with only small and inconsistent gains observed in the one and few-shot learning settings for both PIQA and ARC, but a significant improvement is observed on OpenBookQA. GPT-3 sets SOTA on the new PIQA dataset in all evaluation settings.

3.6 Reading Comprehension

Next we evaluate GPT-3 on the task of reading comprehension. We use a suite of 5 datasets including abstractive, multiple choice, and span based answer formats in both dialog and single question settings. We observe a wide spread in GPT-3’s performance across these datasets suggestive of varying capability with different answer formats. In general we observe GPT-3 is on par with initial baselines and early results trained using contextual representations on each respective dataset.

GPT-3 performs best (within 3 points of the human baseline) on CoQA [RCM19] a free-form conversational dataset and performs worst (13 F1 below an ELMo baseline) on QuAC [CHI⁺18] a dataset which requires modeling structured dialog acts and answer span selections of teacher-student interactions. On DROP [DWD⁺19], a dataset testing discrete reasoning and numeracy in the context of reading comprehension, GPT-3 in a few-shot setting outperforms the fine-tuned BERT baseline from the original paper but is still well below both human performance and state-of-the-art approaches which augment neural networks with symbolic systems [RLL⁺19]. On SQuAD 2.0 [RJL18], GPT-3 demonstrates its few-shot learning capabilities, improving by almost 10 F1 (to 69.8) compared to a zero-shot setting. This allows it to slightly outperform the best fine-tuned result in the original paper. On RACE [LXL⁺17], a multiple choice dataset of middle school and high school english examinations, GPT-3 performs relatively weakly and is only competitive with the earliest work utilizing contextual representations and is still 45% behind SOTA.

3.7 SuperGLUE

In order to better aggregate results on NLP tasks and compare to popular models such as BERT and RoBERTa in a more systematic way, we also evaluate GPT-3 on a standardized collection of datasets, the SuperGLUE benchmark [WPN⁺19] [WPN⁺19] [CLC⁺19] [DMST19] [RBG11] [KCR⁺18] [ZLL⁺18] [DGM06] [BHDD⁺06] [GMDD07] [BDD⁺09] [PCC18] [PHR⁺18]. GPT-3’s test-set performance on the SuperGLUE dataset is shown in Table 3.8. In the few-shot setting, we used 32 examples for all tasks, sampled randomly from the training set. For all tasks except WSC

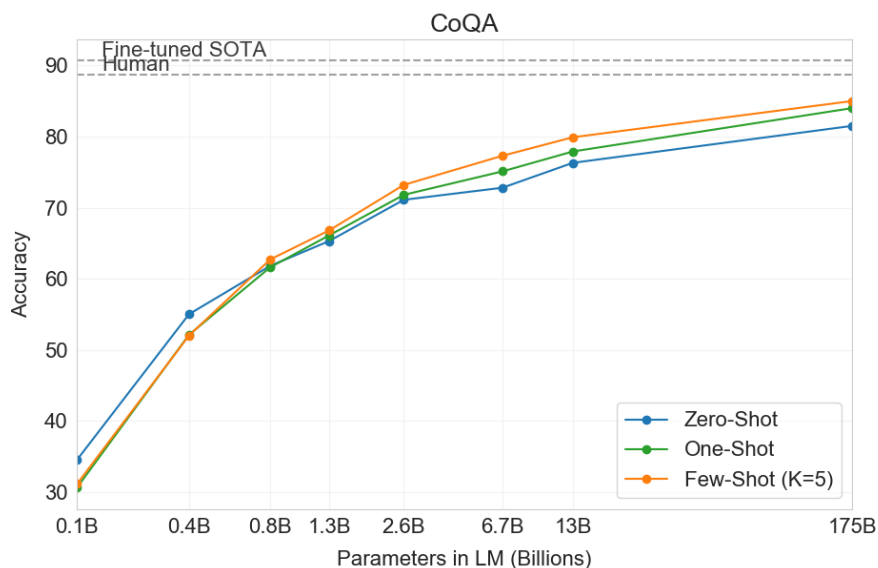


Figure 3.7: GPT-3 results on CoQA reading comprehension task. GPT-3 175B achieves 85 F1 in the few-shot setting, only a few points behind measured human performance and state-of-the-art fine-tuned models. Zero-shot and one-shot performance is a few points behind, with the gains to few-shot being largest for bigger models.

	SuperGLUE Average	BoolQ Accuracy	CB Accuracy	CB F1	COPA Accuracy	RTE Accuracy
Fine-tuned SOTA	89.0	91.0	96.9	93.9	94.8	92.5
Fine-tuned BERT-Large	69.0	77.4	83.6	75.7	70.6	71.7
GPT-3 Few-Shot	71.8	76.4	75.6	52.0	92.0	69.0

	WiC Accuracy	WSC Accuracy	MultiRC Accuracy	MultiRC F1a	ReCoRD Accuracy	ReCoRD F1
Fine-tuned SOTA	76.1	93.8	62.3	88.2	92.5	93.3
Fine-tuned BERT-Large	69.6	64.6	24.1	70.0	71.3	72.0
GPT-3 Few-Shot	49.4	80.1	30.5	75.4	90.2	91.1

Table 3.8: Performance of GPT-3 on SuperGLUE compared to fine-tuned baselines and SOTA. All results are reported on the test set. GPT-3 few-shot is given a total of 32 examples within the context of each task and performs no gradient updates.

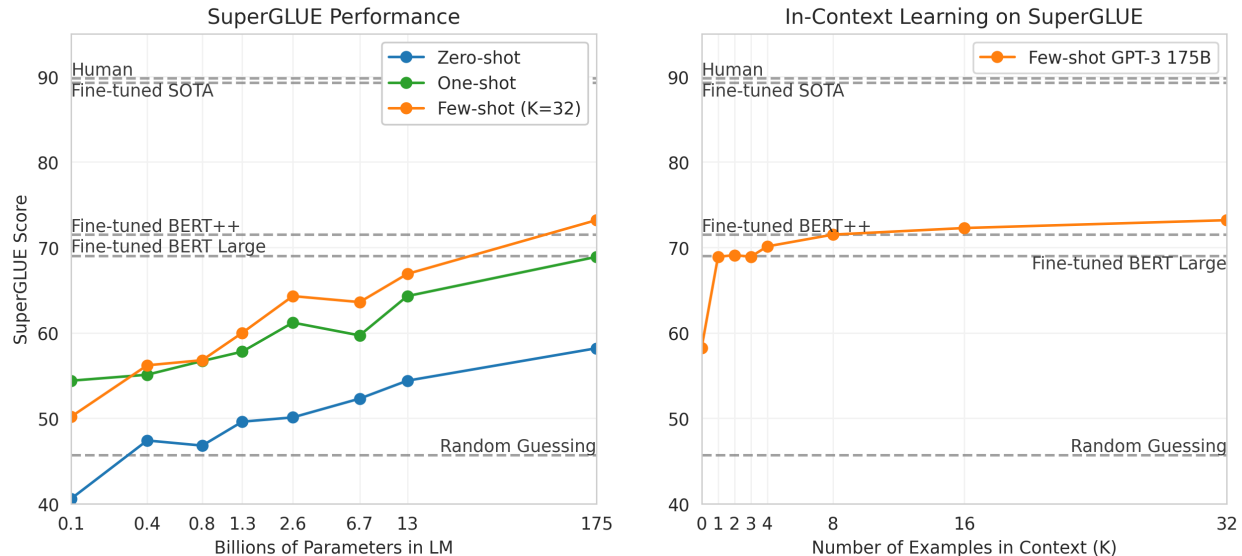


Figure 3.8: Performance on SuperGLUE increases with model size and number of examples in context. A value of $K = 32$ means that our model was shown 32 examples per task, for 256 examples total divided across the 8 tasks in SuperGLUE. We report GPT-3 values on the dev set, so our numbers are not directly comparable to the dotted reference lines (our test set results are in Table 3.8). The BERT-Large reference model was fine-tuned on the SuperGLUE training set (125K examples), whereas BERT++ was first fine-tuned on MultiNLI (392K examples) and SWAG (113K examples) before further fine-tuning on the SuperGLUE training set (for a total of 630K fine-tuning examples). We find the difference in performance between the BERT-Large and BERT++ to be roughly equivalent to the difference between GPT-3 with one example per context versus eight examples per context.

and MultiRC, we sampled a new set of examples to use in the context for each problem. For WSC and MultiRC, we used the same set of randomly drawn examples from the training set as context for all of the problems we evaluated.

We observe a wide range in GPT-3’s performance across tasks. On COPA and ReCoRD GPT-3 achieves near-SOTA performance in the one-shot and few-shot settings, with COPA falling only a couple points short and achieving second place on the leaderboard, where first place is held by a fine-tuned 11 billion parameter model (T5). On WSC, performance is still relatively strong, achieving 80.1% in the few-shot setting (note that GPT-3 achieves 88.6% on the original Winograd dataset as described in Section 3.4). On BoolQ, MultiRC, and RTE, performance is reasonable, roughly matching that of a fine-tuned BERT-Large. On CB, we see signs of life at 75.6% in the few-shot setting.

WiC is a notable weak spot with few-shot performance at 49.4% (at random chance). We tried a number of different phrasings and formulations for WiC (which involves determining if a word is being used with the same meaning in two sentences), none of which was able to achieve strong performance. This hints at a phenomenon that will become clearer in the next section (which discusses the ANLI benchmark) – GPT-3 appears to be weak in the few-shot or one-shot setting at some tasks that involve comparing two sentences or snippets, for example whether a word is used the same way in two sentences (WiC), whether one sentence is a paraphrase of another, or whether one sentence implies another. This could also explain the comparatively low scores for RTE and CB, which also follow this format. Despite these weaknesses, GPT-3 still outperforms a fine-tuned BERT-large on four of eight tasks and on two tasks GPT-3 is close to the state-of-the-art held by a fine-tuned 11 billion parameter model.

Finally, we note that the few-shot SuperGLUE score steadily improves with both model size and with number of examples in the context showing increasing benefits from in-context learning (Figure 3.8). We scale K up to 32 examples per task, after which point additional examples will not reliably fit into our context. When sweeping over values of K , we find that GPT-3 requires less than eight total examples per task to outperform a fine-tuned BERT-Large on overall SuperGLUE score.

3.8 NLI

Natural Language Inference (NLI) [Fyo00] concerns the ability to understand the relationship between two sentences. In practice, this task is usually structured as a two or three class classification problem where the model classifies

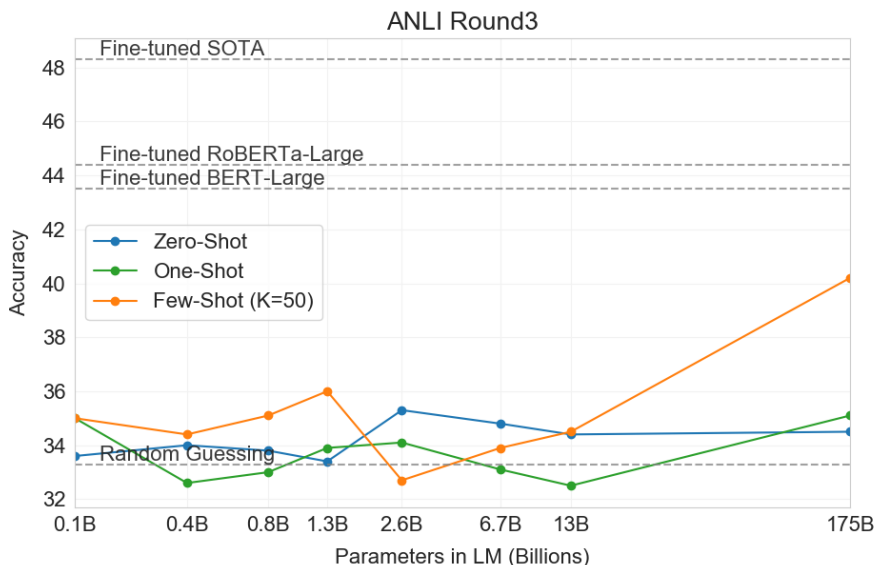


Figure 3.9: Performance of GPT-3 on ANLI Round 3. Results are on the dev-set, which has only 1500 examples and therefore has high variance (we estimate a standard deviation of 1.2%). We find that smaller models hover around random chance, while few-shot GPT-3 175B closes almost half the gap from random chance to SOTA. Results for ANLI rounds 1 and 2 are shown in the appendix.

whether the second sentence logically follows from the first, contradicts the first sentence, or is possibly true (neutral). SuperGLUE includes an NLI dataset, RTE, which evaluates the binary version of the task. On RTE, only the largest version of GPT-3 performs convincingly better than random (56%) in any evaluation setting, but in a few-shot setting GPT-3 performs similarly to a single-task fine-tuned BERT Large. We also evaluate on the recently introduced Adversarial Natural Language Inference (ANLI) dataset [NWD⁺19]. ANLI is a difficult dataset employing a series of adversarially mined natural language inference questions in three rounds (R1, R2, and R3). Similar to RTE, all of our models smaller than GPT-3 perform at almost exactly random chance on ANLI, even in the few-shot setting ($\sim 33\%$), whereas GPT-3 itself shows signs of life on Round 3. Results for ANLI R3 are highlighted in Figure 3.9 and full results for all rounds can be found in Appendix H. These results on both RTE and ANLI suggest that NLI is still a very difficult task for language models and they are only just beginning to show signs of progress.

3.9 Synthetic and Qualitative Tasks

One way to probe GPT-3’s range of abilities in the few-shot (or zero- and one-shot) setting is to give it tasks which require it to perform simple on-the-fly computational reasoning, recognize a novel pattern that is unlikely to have occurred in training, or adapt quickly to an unusual task. We devise several tasks to test this class of abilities. First, we test GPT-3’s ability to perform arithmetic. Second, we create several tasks that involve rearranging or unscrambling the letters in a word, tasks which are unlikely to have been exactly seen during training. Third, we test GPT-3’s ability to solve SAT-style analogy problems few-shot. Finally, we test GPT-3 on several qualitative tasks, including using new words in a sentence, correcting English grammar, and news article generation. We will release the synthetic datasets with the hope of stimulating further study of test-time behavior of language models.

3.9.1 Arithmetic

To test GPT-3’s ability to perform simple arithmetic operations without task-specific training, we developed a small battery of 10 tests that involve asking GPT-3 a simple arithmetic problem in natural language:

- **2 digit addition (2D+)** – The model is asked to add two integers sampled uniformly from $[0, 100)$, phrased in the form of a question, e.g. “Q: What is 48 plus 76? A: 124.”
- **2 digit subtraction (2D-)** – The model is asked to subtract two integers sampled uniformly from $[0, 100)$; the answer may be negative. Example: “Q: What is 34 minus 53? A: -19”.
- **3 digit addition (3D+)** – Same as 2 digit addition, except numbers are uniformly sampled from $[0, 1000)$.

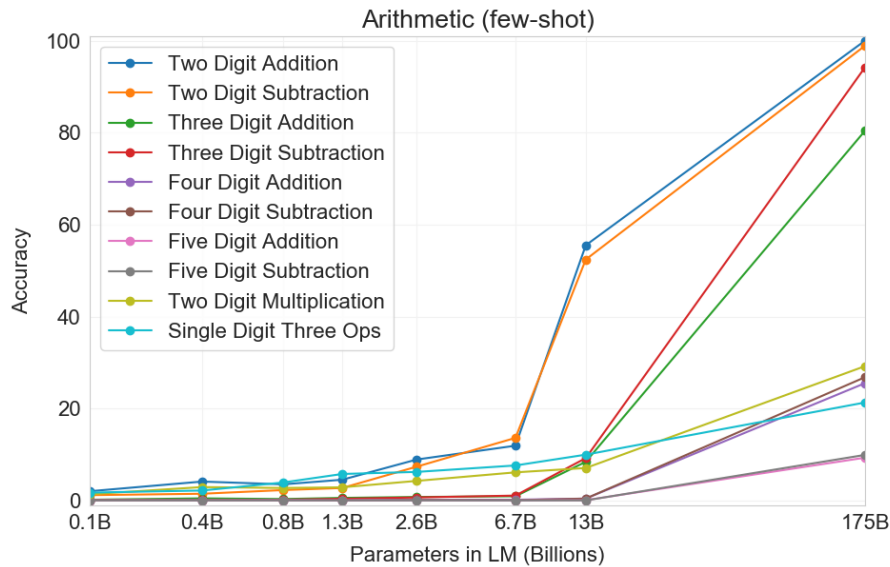


Figure 3.10: Results on all 10 arithmetic tasks in the few-shot settings for models of different sizes. There is a significant jump from the second largest model (GPT-3 13B) to the largest model (GPT-3 175), with the latter being able to reliably accurate 2 digit arithmetic, usually accurate 3 digit arithmetic, and correct answers a significant fraction of the time on 4-5 digit arithmetic, 2 digit multiplication, and compound operations. Results for one-shot and zero-shot are shown in the appendix.

- **3 digit subtraction (3D-)** – Same as 2 digit subtraction, except numbers are uniformly sampled from $[0, 1000)$.
- **4 digit addition (4D+)** – Same as 3 digit addition, except uniformly sampled from $[0, 10000)$.
- **4 digit subtraction (4D-)** – Same as 3 digit subtraction, except uniformly sampled from $[0, 10000)$.
- **5 digit addition (5D+)** – Same as 3 digit addition, except uniformly sampled from $[0, 100000)$.
- **5 digit subtraction (5D-)** – Same as 3 digit subtraction, except uniformly sampled from $[0, 100000)$.
- **2 digit multiplication (2Dx)** – The model is asked to multiply two integers sampled uniformly from $[0, 100)$, e.g. “Q: What is 24 times 42? A: 1008”.
- **One-digit composite (1DC)** – The model is asked to perform a composite operation on three 1 digit numbers, with parentheses around the last two. For example, “Q: What is $6+(4*8)$? A: 38”. The three 1 digit numbers are selected uniformly on $[0, 10)$ and the operations are selected uniformly from $\{+, -, *\}$.

In all 10 tasks the model must generate the correct answer exactly. For each task we generate a dataset of 2,000 random instances of the task and evaluate all models on those instances.

First we evaluate GPT-3 in the few-shot setting, for which results are shown in Figure 3.10. On addition and subtraction, GPT-3 displays strong proficiency when the number of digits is small, achieving 100% accuracy on 2 digit addition, 98.9% at 2 digit subtraction, 80.2% at 3 digit addition, and 94.2% at 3-digit subtraction. Performance decreases as the number of digits increases, but GPT-3 still achieves 25-26% accuracy on four digit operations and 9-10% accuracy on five digit operations, suggesting at least some capacity to generalize to larger numbers of digits. GPT-3 also achieves 29.2% accuracy at 2 digit multiplication, an especially computationally intensive operation. Finally, GPT-3 achieves 21.3% accuracy at single digit combined operations (for example, $9*(7+5)$), suggesting that it has some robustness beyond just single operations.

As Figure 3.10 makes clear, small models do poorly on all of these tasks – even the 13 billion parameter model (the second largest after the 175 billion full GPT-3) can solve 2 digit addition and subtraction only half the time, and all other operations less than 10% of the time.

One-shot and zero-shot performance are somewhat degraded relative to few-shot performance, suggesting that adaptation to the task (or at the very least recognition of the task) is important to performing these computations correctly. Nevertheless, one-shot performance is still quite strong, and even zero-shot performance of the full GPT-3 significantly

Setting	2D+	2D-	3D+	3D-	4D+	4D-	5D+	5D-	2Dx	1DC
GPT-3 Zero-shot	76.9	58.0	34.2	48.3	4.0	7.5	0.7	0.8	19.8	9.8
GPT-3 One-shot	99.6	86.4	65.5	78.7	14.0	14.0	3.5	3.8	27.4	14.3
GPT-3 Few-shot	100.0	98.9	80.4	94.2	25.5	26.8	9.3	9.9	29.2	21.3

Table 3.9: Results on basic arithmetic tasks for GPT-3 175B. $\{2,3,4,5\}D\{+,-\}$ is 2, 3, 4, and 5 digit addition or subtraction, 2Dx is 2 digit multiplication. 1DC is 1 digit composite operations. Results become progressively stronger moving from the zero-shot to one-shot to few-shot setting, but even the zero-shot shows significant arithmetic abilities.

Setting	CL	A1	A2	RI	RW
GPT-3 Zero-shot	3.66	2.28	8.91	8.26	0.09
GPT-3 One-shot	21.7	8.62	25.9	45.4	0.48
GPT-3 Few-shot	37.9	15.1	39.7	67.2	0.44

Table 3.10: GPT-3 175B performance on various word unscrambling and word manipulation tasks, in zero-, one-, and few-shot settings. CL is “cycle letters in word”, A1 is anagrams of but the first and last letters, A2 is anagrams of all but the first and last two letters, RI is “Random insertion in word”, RW is “reversed words”.

outperforms few-shot learning for all smaller models. All three settings for the full GPT-3 are shown in Table 3.9, and model capacity scaling for all three settings is shown in Appendix H.

To spot-check whether the model is simply memorizing specific arithmetic problems, we took the 3-digit arithmetic problems in our test set and searched for them in our training data in both the forms “<NUM1> + <NUM2> =” and “<NUM1> plus <NUM2>”. Out of 2,000 addition problems we found only 17 matches (0.8%) and out of 2,000 subtraction problems we found only 2 matches (0.1%), suggesting that only a trivial fraction of the correct answers could have been memorized. In addition, inspection of incorrect answers reveals that the model often makes mistakes such as not carrying a “1”, suggesting it is actually attempting to perform the relevant computation rather than memorizing a table.

Overall, GPT-3 displays reasonable proficiency at moderately complex arithmetic in few-shot, one-shot, and even zero-shot settings.

3.9.2 Word Scrambling and Manipulation Tasks

To test GPT-3’s ability to learn novel symbolic manipulations from a few examples, we designed a small battery of 5 “character manipulation” tasks. Each task involves giving the model a word distorted by some combination of scrambling, addition, or deletion of characters, and asking it to recover the original word. The 5 tasks are:

- **Cycle letters in word (CL)** – The model is given a word with its letters cycled, then the “=” symbol, and is expected to generate the original word. For example, it might be given “lyinevitab” and should output “inevitably”.
- **Anagrams of all but first and last characters (A1)** – The model is given a word where every letter except the first and last have been scrambled randomly, and must output the original word. Example: criroptuon = corruption.
- **Anagrams of all but first and last 2 characters (A2)** – The model is given a word where every letter except the first 2 and last 2 have been scrambled randomly, and must recover the original word. Example: opoepnnt → opponent.
- **Random insertion in word (RI)** – A random punctuation or space character is inserted between each letter of a word, and the model must output the original word. Example: s.u!c/c!e.s s i/o/n = succession.
- **Reversed words (RW)** – The model is given a word spelled backwards, and must output the original word. Example: stcejbo → objects.

For each task we generate 10,000 examples, which we chose to be the top 10,000 most frequent words as measured by [Nor09] of length more than 4 characters and less than 15 characters. The few-shot results are shown in Figure 3.11. Task performance tends to grow smoothly with model size, with the full GPT-3 model achieving 66.9% on removing

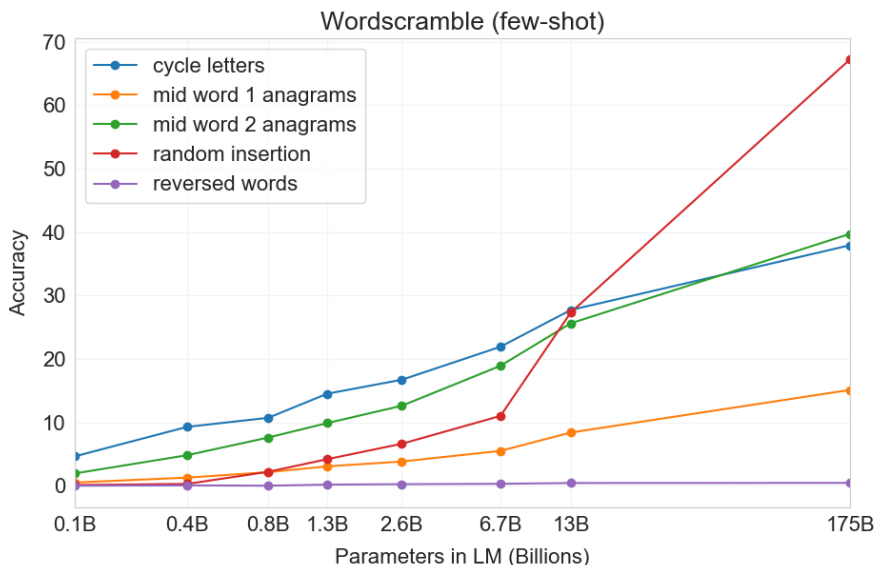


Figure 3.11: Few-shot performance on the five word scrambling tasks for different sizes of model. There is generally smooth improvement with model size although the random insertion task shows an upward slope of improvement with the 175B model solving the majority of the time. Scaling of one-shot and zero-shot performance is shown in the appendix. All tasks are done with $K = 100$.

random insertions, 38.6% on cycling letters, 40.2% on the easier anagram task, and 15.1% on the more difficult anagram task (where only the first and last letters are held fixed). None of the models can reverse the letters in a word.

In the one-shot setting, performance is significantly weaker (dropping by half or more), and in the zero-shot setting the model can rarely perform any of the tasks (Table 3.10). This suggests that the model really does appear to learn these tasks at test time, as the model cannot perform them zero-shot and their artificial nature makes them unlikely to appear in the pre-training data (although we cannot confirm this with certainty).

We can further quantify performance by plotting “in-context learning curves”, which show task performance as a function of the number of in-context examples. We show in-context learning curves for the Symbol Insertion task in Figure 1.2. We can see that larger models are able to make increasingly effective use of in-context information, including both task examples and natural language task descriptions.

Finally, it is worth adding that solving these tasks requires character-level manipulations, whereas our BPE encoding operates on significant fractions of a word (on average ~ 0.7 words per token), so from the LM’s perspective succeeding at these tasks involves not just manipulating BPE tokens but understanding and pulling apart their substructure. Also, CL, A1, and A2 are not bijective (that is, the unscrambled word is not a deterministic function of the scrambled word), requiring the model to perform some search to find the correct unscrambling. Thus, the skills involved appear to require non-trivial pattern-matching and computation.

3.9.3 SAT Analogies

To test GPT-3 on another task that is somewhat unusual relative to the typical distribution of text, we collected a set of 374 “SAT analogy” problems [TLBS03]. Analogies are a style of multiple choice question that constituted a section of the SAT college entrance exam before 2005. A typical example is “audacious is to boldness as (a) sanctimonious is to hypocrisy, (b) anonymous is to identity, (c) remorseful is to misdeed, (d) deleterious is to result, (e) impressionable is to temptation”. The student is expected to choose which of the five word pairs has the same relationship as the original word pair; in this example the answer is “sanctimonious is to hypocrisy”. On this task GPT-3 achieves 65.2% in the few-shot setting, 59.1% in the one-shot setting, and 53.7% in the zero-shot setting, whereas the average score among college applicants was 57% [TL05] (random guessing yields 20%). As shown in Figure 3.12, the results improve with scale, with the the full 175 billion model improving by over 10% compared to the 13 billion parameter model.

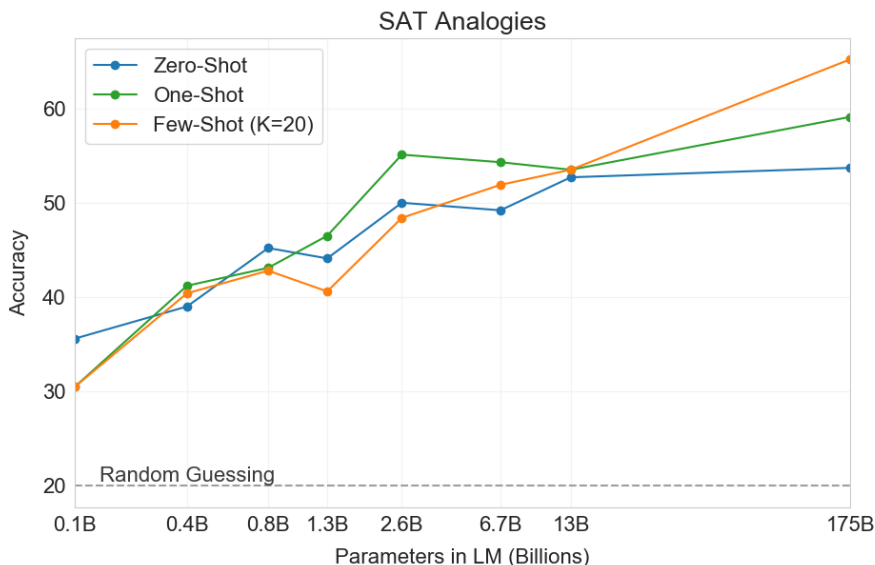


Figure 3.12: Zero-, one-, and few-shot performance on SAT analogy tasks, for different sizes of model. The largest model achieves 65% accuracy in the few-shot setting, and also demonstrates significant gains to in-context learning which are not present in smaller models.

3.9.4 News Article Generation

Previous work on generative language models qualitatively tested their ability to generate synthetic “news articles” by conditional sampling from the model given a human-written prompt consisting of a plausible first sentence for a news story [RWC⁺19]. Relative to [RWC⁺19], the dataset used to train GPT-3 is much less weighted towards news articles, so trying to generate news articles via raw unconditional samples is less effective – for example GPT-3 often interprets the proposed first sentence of a “news article” as a tweet and then posts synthetic responses or follow-up tweets. To solve this problem we employed GPT-3’s few-shot learning abilities by providing three previous news articles in the model’s context to condition it. With the title and subtitle of a proposed next article, the model is able to reliably generate short articles in the “news” genre.

To gauge the quality of news article generation from GPT-3 (which we believe is likely to be correlated with conditional sample generation quality in general), we decided to measure human ability to distinguish GPT-3-generated articles from real ones. Similar work has been carried out by Kreps et al. [KMB20] and Zellers et al. [ZHR⁺19]. Generative language models are trained to match the distribution of content generated by humans, so the (in)ability of humans to distinguish the two is a potentially important measure of quality.³

In order to see how well humans can detect model generated text, we arbitrarily selected 25 article titles and subtitles from the website [newser.com](https://www.newser.com) (mean length: 215 words). We then generated completions of these titles and subtitles from four language models ranging in size from 125M to 175B (GPT-3) parameters (mean length: 200 words). For each model, we presented around 80 US-based participants with a quiz consisting of these real titles and subtitles followed by either the human written article or the article generated by the model⁴. Participants were asked to select whether the article was “very likely written by a human”, “more likely written by a human”, “I don’t know”, “more likely written by a machine”, or “very likely written by a machine”.

The articles we selected were not in the models’ training data and the model outputs were formatted and selected programmatically to prevent human cherry-picking. All models used the same context to condition outputs on and were pre-trained with the same context size and the same article titles and subtitles were used as prompts for each model. However, we also ran an experiment to control for participant effort and attention that followed the same format but involved intentionally bad model generated articles. This was done by generating articles from a “control model”: a 160M parameter model with no context and increased output randomness.

³This task is also relevant to the potential misuse of language models discussed in Section 6.1.

⁴We wanted to identify how good an average person on the internet is at detecting language model outputs, so we focused on participants drawn from the general US population. See Appendix E for details.

	Mean accuracy	95% Confidence Interval (low, hi)	t compared to control (p -value)	“I don’t know” assignments
Control (deliberately bad model)	86%	83%–90%	-	3.6 %
GPT-3 Small	76%	72%–80%	3.9 ($2e-4$)	4.9%
GPT-3 Medium	61%	58%–65%	10.3 ($7e-21$)	6.0%
GPT-3 Large	68%	64%–72%	7.3 ($3e-11$)	8.7%
GPT-3 XL	62%	59%–65%	10.7 ($1e-19$)	7.5%
GPT-3 2.7B	62%	58%–65%	10.4 ($5e-19$)	7.1%
GPT-3 6.7B	60%	56%–63%	11.2 ($3e-21$)	6.2%
GPT-3 13B	55%	52%–58%	15.3 ($1e-32$)	7.1%
GPT-3 175B	52%	49%–54%	16.9 ($1e-34$)	7.8%

Table 3.11: Human accuracy in identifying whether short (~ 200 word) news articles are model generated. We find that human accuracy (measured by the ratio of correct assignments to non-neutral assignments) ranges from 86% on the control model to 52% on GPT-3 175B. This table compares mean accuracy between five different models, and shows the results of a two-sample T-Test for the difference in mean accuracy between each model and the control model (an unconditional GPT-3 Small model with increased output randomness).

Mean human accuracy (the ratio of correct assignments to non-neutral assignments per participant) at detecting that the intentionally bad articles were model generated was $\sim 86\%$ where 50% is chance level performance. By contrast, mean human accuracy at detecting articles that were produced by the 175B parameter model was barely above chance at $\sim 52\%$ (see Table 3.11).⁵ Human abilities to detect model generated text appear to decrease as model size increases: there appears to be a trend towards chance accuracy with model size, and human detection of GPT-3 is close to chance.⁶ This is true despite the fact that participants spend more time on each output as model size increases (see Appendix E).

Examples of synthetic articles from GPT-3 are given in Figures 3.14 and 3.15.⁷ Much of the text is—as indicated by the evaluations—difficult for humans to distinguish from authentic human content. Factual inaccuracies can be an indicator that an article is model generated since, unlike human authors, the models have no access to the specific facts that the article titles refer to or when the article was written. Other indicators include repetition, non sequiturs, and unusual phrasings, though these are often subtle enough that they are not noticed.

Related work on language model detection by Ippolito et al. [IDCBE19] indicates that automatic discriminators like GROVER [ZHR⁺19] and GLTR [GSR19] may have greater success at detecting model generated text than human evaluators. Automatic detection of these models may be a promising area of future research.

Ippolito et al. [IDCBE19] also note that human accuracy at detecting model generated text increases as humans observe more tokens. To do a preliminary investigation of how good humans are at detecting longer news articles generated by GPT-3 175B, we selected 12 world news articles from Reuters with an average length of 569 words and generated completions of these articles from GPT-3 with an average length of 498 words (298 words longer than our initial experiments). Following the methodology above, we ran two experiments, each on around 80 US-based participants, to compare human abilities to detect the articles generated by GPT-3 and a control model.

We found that mean human accuracy at detecting the intentionally bad longer articles from the control model was $\sim 88\%$, while mean human accuracy at detecting the longer articles that were produced by GPT-3 175B was still barely above chance at $\sim 52\%$ (see Table 3.12). This indicates that, for news articles that are around 500 words long, GPT-3 continues to produce articles that humans find difficult to distinguish from human written news articles.

3.9.5 Learning and Using Novel Words

A task studied in developmental linguistics [CB78] is the ability to learn and utilize new words, for example using a word in a sentence after seeing it defined only once, or conversely inferring a word’s meaning from only one usage. Here we qualitatively test GPT-3’s ability to do the former. Specifically, we give GPT-3 the definition of a nonexistent word, such as “Gigamuru”, and then ask it to use it in a sentence. We provide one to five previous examples of a (separate)

⁵We use a two-sample Student’s T-Test to test for significant difference between the means of the participant accuracies of each model and the control model and report the normalized difference in the means (as the t -statistic) and the p -value.

⁶If a model consistently produces texts that are more impressive than human articles, it is possible that human performance on this task would drop below 50%. Indeed, many individual participants scored below 50% on this task.

⁷Additional non-news samples can be found in Appendix F.

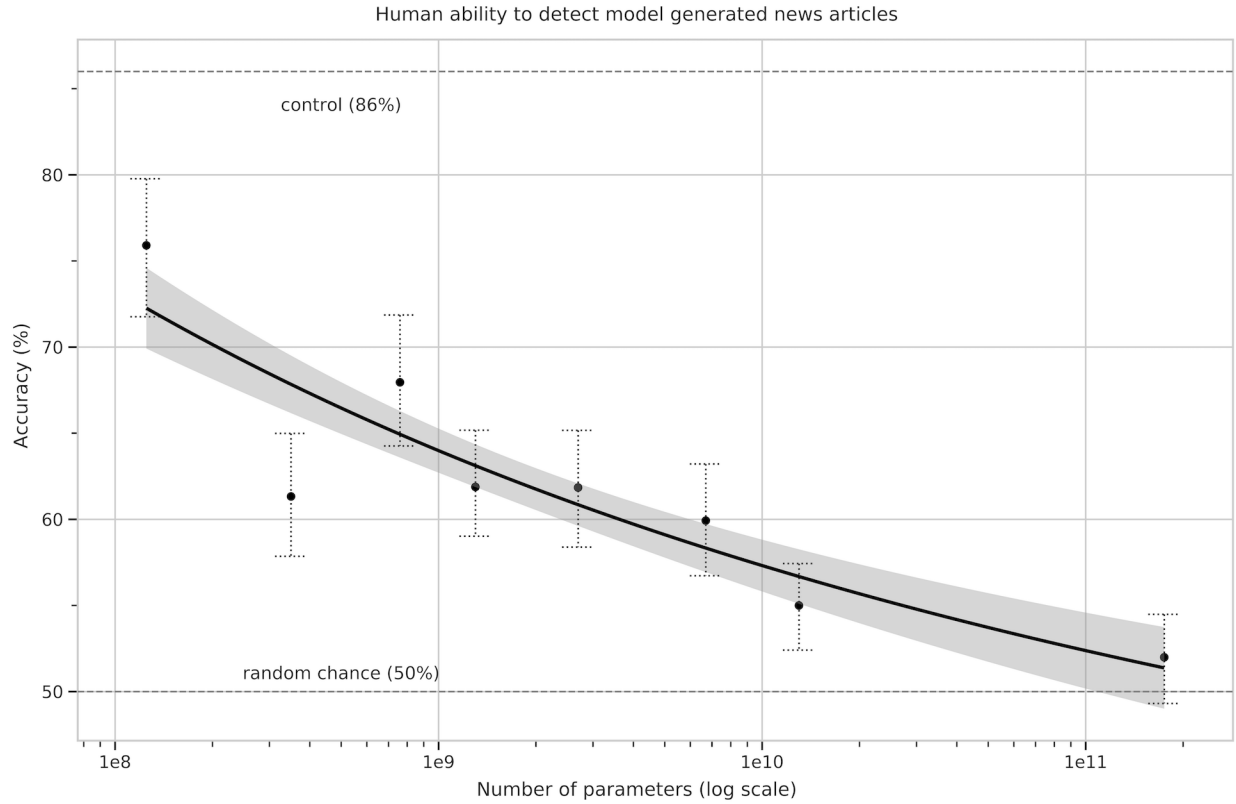


Figure 3.13: People’s ability to identify whether news articles are model-generated (measured by the ratio of correct assignments to non-neutral assignments) decreases as model size increases. Accuracy on the outputs on the deliberately-bad control model (an unconditioned GPT-3 Small model with higher output randomness) is indicated with the dashed line at the top, and the random chance (50%) is indicated with the dashed line at the bottom. Line of best fit is a power law with 95% confidence intervals.

	Mean accuracy	95% Confidence Interval (low, hi)	<i>t</i> compared to control (<i>p</i> -value)	“I don’t know” assignments
Control	88%	84%–91%	-	2.7%
GPT-3 175B	52%	48%–57%	12.7 (3.2e-23)	10.6%

Table 3.12: People’s ability to identify whether ~ 500 word articles are model generated (as measured by the ratio of correct assignments to non-neutral assignments) was 88% on the control model and 52% on GPT-3 175B. This table shows the results of a two-sample T-Test for the difference in mean accuracy between GPT-3 175B and the control model (an unconditional GPT-3 Small model with increased output randomness).

Title: United Methodists Agree to Historic Split
 Subtitle: Those who oppose gay marriage will form their own denomination
 Article: After two days of intense debate, the United Methodist Church has agreed to a historic split - one that is expected to end in the creation of a new denomination, one that will be "theologically and socially conservative," according to The Washington Post. The majority of delegates attending the church's annual General Conference in May voted to strengthen a ban on the ordination of LGBTQ clergy and to write new rules that will "discipline" clergy who officiate at same-sex weddings. But those who opposed these measures have a new plan: They say they will form a separate denomination by 2020, calling their church the Christian Methodist denomination.

The Post notes that the denomination, which claims 12.5 million members, was in the early 20th century the "largest Protestant denomination in the U.S.," but that it has been shrinking in recent decades. The new split will be the second in the church's history. The first occurred in 1968, when roughly 10 percent of the denomination left to form the Evangelical United Brethren Church. The Post notes that the proposed split "comes at a critical time for the church, which has been losing members for years," which has been "pushed toward the brink of a schism over the role of LGBTQ people in the church." Gay marriage is not the only issue that has divided the church. In 2016, the denomination was split over ordination of transgender clergy, with the North Pacific regional conference voting to ban them from serving as clergy, and the South Pacific regional conference voting to allow them.

Figure 3.14: The GPT-3 generated news article that humans had the greatest difficulty distinguishing from a human written article (accuracy: 12%).

Title: Star's Tux Promise Draws Megyn Kelly's Sarcasm
 Subtitle: Joaquin Phoenix pledged to not change for each awards event
 Article: A year ago, Joaquin Phoenix made headlines when he appeared on the red carpet at the Golden Globes wearing a tuxedo with a paper bag over his head that read, "I am a shape-shifter. I can't change the world. I can only change myself." It was a promise to not change to fit into the Hollywood mold: "I think that's a really special thing, to not change yourself. I think it's a really special thing to say, 'This is what's inside of me, I'm proud of it, and I'm not going to be ashamed because of the way that someone else thinks I should be.'" Now, it's the Oscars, and Phoenix is at it again. But this time, his publicist is saying he'll be wearing a tux no matter what.

Megyn Kelly was not impressed, and she let him have it on The Tonight Show. "You know, I feel like, I feel like you could have worn the tux," she says. "But you're saying you're a shape-shifter. I don't know if you can change your tux, but you can change your mind. You can change your mind. You can change your mind." Phoenix says he did, but it didn't stick. "I was like, 'Okay, I'm going to wear a tuxedo to this thing.' And then I thought, 'I don't want to wear a tuxedo to this thing.'" Kelly goes on to encourage him to change his mind again, but Phoenix says it's too late: "I'm committed to wearing this."

Figure 3.15: The GPT-3 generated news article that humans found the easiest to distinguish from a human written article (accuracy: 61%).

A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word whatpu is:
 We were traveling in Africa and we saw these very cute whatpus.

To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word farduddle is:
One day when I was playing tag with my little sister, she got really excited and she started doing these crazy farduddles.

A "yalubalu" is a type of vegetable that looks like a big pumpkin. An example of a sentence that uses the word yalubalu is:
I was on a trip to Africa and I tried this yalubalu vegetable that was grown in a garden there. It was delicious.

A "Burring" is a car with very fast acceleration. An example of a sentence that uses the word Burring is:
In our garage we have a Burring that my father drives to work every day.

A "Gigamuru" is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru is:
I have a Gigamuru that my uncle gave me as a gift. I love to play it at home.

To "screeg" something is to swing a sword at it. An example of a sentence that uses the word screeg is:
We screeghed at each other for several minutes and then we went outside and ate ice cream.

Figure 3.16: Representative GPT-3 completions for the few-shot task of using a new word in a sentence. Boldface is GPT-3’s completions, plain text is human prompts. In the first example both the prompt and the completion are provided by a human; this then serves as conditioning for subsequent examples where GPT-3 receives successive additional prompts and provides the completions. Nothing task-specific is provided to GPT-3 other than the conditioning shown here.

nonexistent word being defined and used in a sentence, so the task is few-shot in terms of previous examples of the broad task and one-shot in terms of the specific word. Table 3.16 shows the 6 examples we generated; all definitions were human-generated, and the first answer was human-generated as conditioning while the subsequent answers were generated by GPT-3. These examples were generated continuously in one sitting and we did not omit or repeatedly try any prompts. In all cases the generated sentence appears to be a correct or at least plausible use of the word. In the final sentence the model generates a plausible conjugation for the word “screeg” (namely “screeghed”), although the use of the word is slightly awkward (“screeghed at each other”) despite being plausible in the sense that it could describe a toy sword fight. Overall, GPT-3 appears to be at least proficient at the task of using novel words in a sentence.

3.9.6 Correcting English Grammar

Another task well suited for few-shot learning is correcting English grammar. We test this with GPT-3 in the few-shot setting by giving prompts of the form "Poor English Input: <sentence>\n Good English Output: <sentence>". We give GPT-3 one human-generated correction and then ask it to correct 5 more (again without any omissions or repeats). Results are shown in Figure 3.17.

4 Measuring and Preventing Memorization Of Benchmarks

Since our training dataset is sourced from the internet, it is possible that our model was trained on some of our benchmark test sets. Accurately detecting test contamination from internet-scale datasets is a new area of research without established best practices. While it is common practice to train large models without investigating contamination, given the increasing scale of pretraining datasets, we believe this issue is becoming increasingly important to attend to.

This concern is not just hypothetical. One of the first papers to train a language model on Common Crawl data [TL18] detected and removed a training document which overlapped with one of their evaluation datasets. Other work such as GPT-2 [RWC⁺19] also conducted post-hoc overlap analysis. Their study was relatively encouraging, finding that

Poor English input:	I eated the purple berries.
Good English output:	I ate the purple berries.
Poor English input:	Thank you for picking me as your designer. I'd appreciate it.
Good English output:	Thank you for choosing me as your designer. I appreciate it.
Poor English input:	The mentioned changes have done. or I did the alteration that you requested. or I changed things you wanted and did the modifications.
Good English output:	The requested changes have been made. or I made the alteration that you requested. or I changed things you wanted and made the modifications.
Poor English input:	I'd be more than happy to work with you in another project.
Good English output:	I'd be more than happy to work with you on another project.
Poor English input:	Please provide me with a short brief of the design you're looking for and that'd be nice if you could share some examples or project you did before.
Good English output:	Please provide me with a brief description of the design you're looking for and that would be nice if you could share some examples or projects you have done before.
Poor English input:	The patient was died.
Good English output:	The patient died.
Poor English input:	We think that Leslie likes ourselves.
Good English output:	We think that Leslie likes us.
Poor English input:	Janet broke Bill on the finger.
Good English output:	Janet broke Bill's finger.
Poor English input:	Mary arranged for, in St. Louis, John to rent a house cheap.
Good English output:	Mary arranged for John to rent a house in St. Louis.
Poor English input:	Today I have went to the store to to buys some many bottle of water.
Good English output:	Today I went to the store to buy some bottles of water.
Poor English input:	I have tried to hit ball with bat, but my swing is has miss.
Good English output:	I tried to hit the ball with the bat, but my swing missed.

Figure 3.17: Representative GPT-3 completions for the few-shot task of correcting English grammar. Boldface is GPT-3's completions, plain text is human prompts. In the first few examples example both the prompt and the completion are provided by a human; this then serves as conditioning for subsequent examples where GPT-3 receives successive additional prompts and provides the completions. Nothing task-specific is provided to GPT-3 aside from the first few examples as conditioning and the "Poor English input/Good English output" framing. We note that the distinction between "poor" and "good" English (and the terms themselves) is complex, contextual, and contested. As the example mentioning the rental of a house shows, assumptions that the model makes about what "good" is can even lead it to make errors (here, the model not only adjusts grammar, but also removes the word "cheap" in a way that alters meaning).

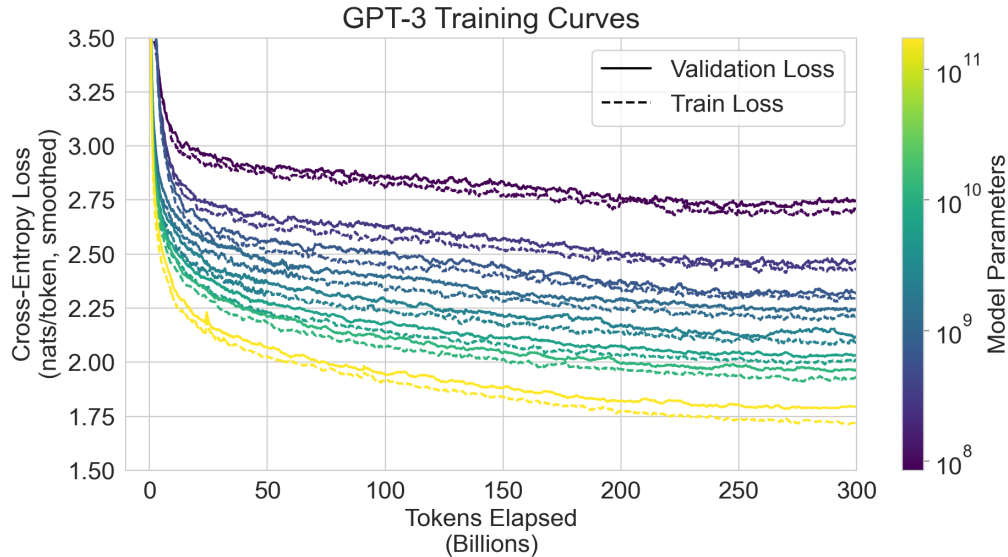


Figure 4.1: GPT-3 Training Curves We measure model performance during training on a deduplicated validation split of our training distribution. Though there is some gap between training and validation performance, the gap grows only minimally with model size and training time, suggesting that most of the gap comes from a difference in difficulty rather than overfitting.

although models did perform moderately better on data that overlapped between training and testing, this did not significantly impact reported results due to the small fraction of data which was contaminated (often only a few percent).

GPT-3 operates in a somewhat different regime. On the one hand, the dataset and model size are about two orders of magnitude larger than those used for GPT-2, and include a large amount of Common Crawl, creating increased potential for contamination and memorization. On the other hand, precisely due to the large amount of data, even GPT-3 175B does not overfit its training set by a significant amount, measured relative to a held-out validation set with which it was deduplicated (Figure 4.1). Thus, we expect that contamination is likely to be frequent, but that its effects may not be as large as feared.

We initially tried to address the issue of contamination by proactively searching for and attempting to remove any overlap between our training data and the development and test sets of all benchmarks studied in this paper. Unfortunately, a bug resulted in only partial removal of all detected overlaps from the training data. Due to the cost of training, it wasn’t feasible to retrain the model. To address this, we investigate in detail how the remaining detected overlap impacts results.

For each benchmark, we produce a ‘clean’ version which removes all potentially leaked examples, defined roughly as examples that have a 13-gram overlap with anything in the pretraining set (or that overlap with the whole example when it is shorter than 13-grams). The goal is to very conservatively flag anything that could potentially be contamination, so as to produce a clean subset that is free of contamination with high confidence. The exact procedure is detailed in Appendix C.

We then evaluate GPT-3 on these clean benchmarks, and compare to the original score. If the score on the clean subset is similar to the score on the entire dataset, this suggests that contamination, even if present, does not have a significant effect on reported results. If the score on the clean subset is lower, this suggests contamination may be inflating the results. The results are summarized in Figure 4.2. Although potential contamination is often high (with a quarter of benchmarks scoring over 50%), in most cases performance changes only negligibly, and we see no evidence that contamination level and performance difference are correlated. We conclude that either our conservative method substantially overestimated contamination or that contamination has little effect on performance.

Below, we review in more detail the few specific cases where either (1) the model performs significantly worse on the cleaned version, or (2) potential contamination is very high, which makes measuring the performance difference difficult.

Our analysis flagged six groups of benchmarks for further investigation: Word Scrambling, Reading Comprehension (QuAC, SQuAD2, DROP), PIQA, Winograd, language modeling tasks (Wikitext tasks, IBW), and German to English

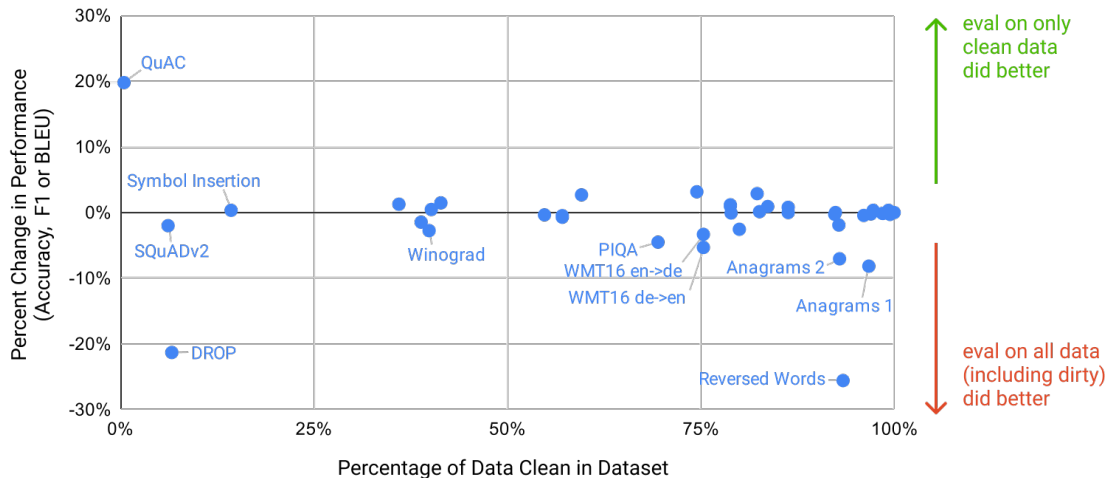


Figure 4.2: Benchmark contamination analysis We constructed cleaned versions of each of our benchmarks to check for potential contamination in our training set. The x-axis is a conservative lower bound for how much of the dataset is known with high confidence to be clean, and the y-axis shows the difference in performance when evaluating only on the verified clean subset. Performance on most benchmarks changed negligibly, but some were flagged for further review. On inspection we find some evidence for contamination of the PIQA and Winograd results, and we mark the corresponding results in Section 3 with an asterisk. We find no evidence that other benchmarks are affected.

translation. Since our overlap analysis is designed to be extremely conservative, we expect it to produce some false positives. We summarize the results for each group of tasks below:

- **Reading Comprehension:** Our initial analysis flagged >90% of task examples from QuAC, SQuAD2, and DROP as potentially contaminated, so large that even measuring the differential on a clean subset was difficult. Upon manual inspection, however, we found that for every overlap we inspected, in all 3 datasets, the source text was present in our training data but the question/answer pairs were not, meaning the model gains only background information and cannot memorize the answer to a specific question.
- **German translation:** We found 25% of the examples in the WMT16 German-English test set were marked as potentially contaminated, with an associated total effect size of 1-2 BLEU. Upon inspection, none of the flagged examples contain paired sentences resembling NMT training data and collisions were monolingual matches mostly of snippets of events discussed in the news.
- **Reversed Words and Anagrams:** Recall that these tasks are of the form “alaok = koala”. Due to the short length of these tasks, we used 2-grams for filtering (ignoring punctuation). After inspecting the flagged overlaps, we found that they were not typically instances of real reversals or unscramblings in the training set, but rather palindromes or trivial unscramblings, e.g “kayak = kayak”. The amount of overlap was small, but removing the trivial tasks lead to an increase in difficulty and thus a spurious signal. Related to this, the symbol insertion task shows high overlap but no effect on performance – this is because that task involves removing non-letter characters from a word, and the overlap analysis itself ignores such characters, leading to many spurious matches.
- **PIQA:** The overlap analysis flagged 29% of examples as contaminated, and observed a 3 percentage point absolute decrease (4% relative decrease) in performance on the clean subset. Though the test dataset was released after our training set was created and its labels are hidden, some of the web pages used by the crowdsourced dataset creators are contained in our training set. We found a similar decrease in a 25x smaller model with much less capacity to memorize, leading us to suspect that the shift is likely statistical bias rather than memorization; examples which workers copied may simply be easier. Unfortunately, we cannot rigorously prove this hypothesis. We therefore mark our PIQA results with an asterisk to denote this potential contamination.
- **Winograd:** The overlap analysis flagged 45% of examples, and found a 2.6% decrease in performance on the clean subset. Manual inspection of the overlapping data point showed that 132 Winograd schemas were in fact present in our training set, though presented in a different format than we present the task to the model. Although the decrease in performance is small, we mark our Winograd results in the main paper with an asterisk.

- **Language modeling:** We found the 4 Wikipedia language modeling benchmarks measured in GPT-2, plus the Children’s Book Test dataset, to be almost entirely contained in our training data. Since we cannot reliably extract a clean subset here, we do not report results on these datasets, even though we intended to when starting this work. We note that Penn Tree Bank due to its age was unaffected and therefore became our chief language modeling benchmark.

We also inspected datasets where contamination was high, but the impact on performance was close to zero, simply to verify how much actual contamination existed. These appeared to often contain false positives. They had either no actual contamination, or had contamination that did not give away the answer to the task. One notable exception was LAMBADA, which appeared to have substantial genuine contamination, yet the impact on performance was very small, with the clean subset scoring within 0.5% of the full dataset. Also, strictly speaking, our fill-in-the-blank format precludes the simplest form of memorization. Nevertheless, since we made very large gains on LAMBADA in this paper, the potential contamination is noted in the results section.

An important limitation of our contamination analysis is that we cannot be sure that the clean subset is drawn from the same distribution as the original dataset. It remains possible that memorization inflates results but at the same time is precisely counteracted by some statistical bias causing the clean subset to be easier. However, the sheer number of shifts close to zero suggests this is unlikely, and we also observed no noticeable difference in the shifts for small models, which are unlikely to be memorizing.

Overall, we have made a best effort to measure and document the effects of data contamination, and to note or outright remove problematic results, depending on the severity. Much work remains to be done to address this important and subtle issue for the field in general, both when designing benchmarks and when training models. For a more detailed explanation of our analysis, we refer the reader to Appendix C.

5 Limitations

GPT-3 and our analysis of it have a number of limitations. Below we describe some of these and suggest directions for future work.

First, despite the strong quantitative and qualitative improvements of GPT-3, particularly compared to its direct predecessor GPT-2, it still has notable weaknesses in text synthesis and several NLP tasks. On text synthesis, although the overall quality is high, GPT-3 samples still sometimes repeat themselves semantically at the document level, start to lose coherence over sufficiently long passages, contradict themselves, and occasionally contain non-sequitur sentences or paragraphs. We will release a collection of 500 uncurated unconditional samples to help provide a better sense of GPT-3’s limitations and strengths at text synthesis. Within the domain of discrete language tasks, we have noticed informally that GPT-3 seems to have special difficulty with “common sense physics”, despite doing well on some datasets (such as PIQA [BZB⁺19]) that test this domain. Specifically GPT-3 has difficulty with questions of the type “If I put cheese into the fridge, will it melt?”. Quantitatively, GPT-3’s in-context learning performance has some notable gaps on our suite of benchmarks, as described in Section 3, and in particular it does little better than chance when evaluated one-shot or even few-shot on some “comparison” tasks, such as determining if two words are used the same way in a sentence, or if one sentence implies another (WIC and ANLI respectively), as well as on a subset of reading comprehension tasks. This is especially striking given GPT-3’s strong few-shot performance on many other tasks.

GPT-3 has several structural and algorithmic limitations, which could account for some of the issues above. We focused on exploring in-context learning behavior in autoregressive language models because it is straightforward to both sample and compute likelihoods with this model class. As a result our experiments do not include any bidirectional architectures or other training objectives such as denoising. This is a noticeable difference from much of the recent literature, which has documented improved fine-tuning performance when using these approaches over standard language models [RSR⁺19]. Thus our design decision comes at the cost of potentially worse performance on tasks which empirically benefit from bidirectionality. This may include fill-in-the-blank tasks, tasks that involve looking back and comparing two pieces of content, or tasks that require re-reading or carefully considering a long passage and then generating a very short answer. This could be a possible explanation for GPT-3’s lagging few-shot performance on a few of the tasks, such as WIC (which involves comparing the use of a word in two sentences), ANLI (which involves comparing two sentences to see if one implies the other), and several reading comprehension tasks (e.g. QuAC and RACE). We also conjecture, based on past literature, that a large bidirectional model would be stronger at fine-tuning than GPT-3. Making a bidirectional model at the scale of GPT-3, and/or trying to make bidirectional models work with few- or zero-shot learning, is a promising direction for future research, and could help achieve the “best of both worlds”.

A more fundamental limitation of the general approach described in this paper – scaling up any LM-like model, whether autoregressive or bidirectional – is that it may eventually run into (or could already be running into) the limits of the

pretraining objective. Our current objective weights every token equally and lacks a notion of what is most important to predict and what is less important. [RRS20] demonstrate benefits of customizing prediction to entities of interest. Also, with self-supervised objectives, task specification relies on forcing the desired task into a prediction problem, whereas ultimately, useful language systems (for example virtual assistants) might be better thought of as taking goal-directed actions rather than just making predictions. Finally, large pretrained language models are not grounded in other domains of experience, such as video or real-world physical interaction, and thus lack a large amount of context about the world [BHT⁺20]. For all these reasons, scaling pure self-supervised prediction is likely to hit limits, and augmentation with a different approach is likely to be necessary. Promising future directions in this vein might include learning the objective function from humans [ZSW⁺19a], fine-tuning with reinforcement learning, or adding additional modalities such as images to provide grounding and a better model of the world [CLY⁺19].

Another limitation broadly shared by language models is poor sample efficiency during pre-training. While GPT-3 takes a step towards test-time sample efficiency closer to that of humans (one-shot or zero-shot), it still sees much more text during pre-training than a human sees in their lifetime [Lin20]. Improving pre-training sample efficiency is an important direction for future work, and might come from grounding in the physical world to provide additional information, or from algorithmic improvements.

A limitation, or at least uncertainty, associated with few-shot learning in GPT-3 is ambiguity about whether few-shot learning actually learns new tasks “from scratch” at inference time, or if it simply recognizes and identifies tasks that it has learned during training. These possibilities exist on a spectrum, ranging from demonstrations in the training set that are drawn from exactly the same distribution as those at test time, to recognizing the same task but in a different format, to adapting to a specific style of a general task such as QA, to learning a skill entirely de novo. Where GPT-3 is on this spectrum may also vary from task to task. Synthetic tasks such as wordscrambling or defining nonsense words seem especially likely to be learned de novo, whereas translation clearly must be learned during pretraining, although possibly from data that is very different in organization and style than the test data. Ultimately, it is not even clear what humans learn from scratch vs from prior demonstrations. Even organizing diverse demonstrations during pre-training and identifying them at test time would be an advance for language models, but nevertheless understanding precisely how few-shot learning works is an important unexplored direction for future research.

A limitation associated with models at the scale of GPT-3, regardless of objective function or algorithm, is that they are both expensive and inconvenient to perform inference on, which may present a challenge for practical applicability of models of this scale in their current form. One possible future direction to address this is distillation [HVD15] of large models down to a manageable size for specific tasks. Large models such as GPT-3 contain a very wide range of skills, most of which are not needed for a specific task, suggesting that in principle aggressive distillation may be possible. Distillation is well-explored in general [LHCG19a] but has not been tried at the scale of hundred of billions of parameters; new challenges and opportunities may be associated with applying it to models of this size.

Finally, GPT-3 shares some limitations common to most deep learning systems – its decisions are not easily interpretable, it is not necessarily well-calibrated in its predictions on novel inputs as observed by the much higher variance in performance than humans on standard benchmarks, and it retains the biases of the data it has been trained on. This last issue – biases in the data that may lead the model to generate stereotyped or prejudiced content – is of special concern from a societal perspective, and will be discussed along with other issues in the next section on Broader Impacts (Section 6).

6 Broader Impacts

Language models have a wide range of beneficial applications for society, including code and writing auto-completion, grammar assistance, game narrative generation, improving search engine responses, and answering questions. But they also have potentially harmful applications. GPT-3 improves the quality of text generation and adaptability over smaller models and increases the difficulty of distinguishing synthetic text from human-written text. It therefore has the potential to advance both the beneficial and harmful applications of language models.

Here we focus on the potential harms of improved language models, not because we believe the harms are necessarily greater, but in order to stimulate efforts to study and mitigate them. The broader impacts of language models like this are numerous. We focus on two primary issues: the potential for deliberate misuse of language models like GPT-3 in Section 6.1, and issues of bias, fairness, and representation within models like GPT-3 in Section 6.2. We also briefly discuss issues of energy efficiency (Section 6.3).

6.1 Misuse of Language Models

Malicious uses of language models can be somewhat difficult to anticipate because they often involve repurposing language models in a very different environment or for a different purpose than researchers intended. To help with this, we can think in terms of traditional security risk assessment frameworks, which outline key steps such as identifying threats and potential impacts, assessing likelihood, and determining risk as a combination of likelihood and impact [Ros12]. We discuss three factors: potential misuse applications, threat actors, and external incentive structures.

6.1.1 Potential Misuse Applications

Any socially harmful activity that relies on generating text could be augmented by powerful language models. Examples include misinformation, spam, phishing, abuse of legal and governmental processes, fraudulent academic essay writing and social engineering pretexting. Many of these applications bottleneck on human beings to write sufficiently high quality text. Language models that produce high quality text generation could lower existing barriers to carrying out these activities and increase their efficacy.

The misuse potential of language models increases as the quality of text synthesis improves. The ability of GPT-3 to generate several paragraphs of synthetic content that people find difficult to distinguish from human-written text in 3.9.4 represents a concerning milestone in this regard.

6.1.2 Threat Actor Analysis

Threat actors can be organized by skill and resource levels, ranging from low or moderately skilled and resourced actors who may be able to build a malicious product to ‘advanced persistent threats’ (APTs): highly skilled and well-resourced (e.g. state-sponsored) groups with long-term agendas [SBC⁺19].

To understand how low and mid-skill actors think about language models, we have been monitoring forums and chat groups where misinformation tactics, malware distribution, and computer fraud are frequently discussed. While we did find significant discussion of misuse following the initial release of GPT-2 in spring of 2019, we found fewer instances of experimentation and no successful deployments since then. Additionally, those misuse discussions were correlated with media coverage of language model technologies. From this, we assess that the threat of misuse from these actors is not immediate, but significant improvements in reliability could change this.

Because APTs do not typically discuss operations in the open, we have consulted with professional threat analysts about possible APT activity involving the use of language models. Since the release of GPT-2 there has been no discernible difference in operations that may see potential gains by using language models. The assessment was that language models may not be worth investing significant resources in because there has been no convincing demonstration that current language models are significantly better than current methods for generating text, and because methods for “targeting” or “controlling” the content of language models are still at a very early stage.

6.1.3 External Incentive Structures

Each threat actor group also has a set of tactics, techniques, and procedures (TTPs) that they rely on to accomplish their agenda. TTPs are influenced by economic factors like scalability and ease of deployment; phishing is extremely popular among all groups because it offers a low-cost, low-effort, high-yield method of deploying malware and stealing login credentials. Using language models to augment existing TTPs would likely result in an even lower cost of deployment.

Ease of use is another significant incentive. Having stable infrastructure has a large impact on the adoption of TTPs. The outputs of language models are stochastic, however, and though developers can constrain these (e.g. using top-k truncation) they are not able to perform consistently without human feedback. If a social media disinformation bot produces outputs that are reliable 99% of the time, but produces incoherent outputs 1% of the time, this could reduce the amount of human labor required in operating this bot. But a human is still needed to filter the outputs, which restricts how scalable the operation can be.

Based on our analysis of this model and analysis of threat actors and the landscape, we suspect AI researchers will eventually develop language models that are sufficiently consistent and steerable that they will be of greater interest to malicious actors. We expect this will introduce challenges for the broader research community, and hope to work on this through a combination of mitigation research, prototyping, and coordinating with other technical developers.

6.2 Fairness, Bias, and Representation

Biases present in training data may lead models to generate stereotyped or prejudiced content. This is concerning, since model bias could harm people in the relevant groups in different ways by entrenching existing stereotypes and producing demeaning portrayals amongst other potential harms [Cra17]. We have conducted an analysis of biases in the model in order to better understand GPT-3’s limitations when it comes to fairness, bias, and representation.⁸

Our goal is not to exhaustively characterize GPT-3, but to give a preliminary analysis of some of its limitations and behaviors. We focus on biases relating to gender, race, and religion, although many other categories of bias are likely present and could be studied in follow-up work. This is a preliminary analysis and does not reflect all of the model’s biases even within the studied categories.

Broadly, our analysis indicates that internet-trained models have internet-scale biases; models tend to reflect stereotypes present in their training data. Below we discuss our preliminary findings of bias along the dimensions of gender, race, and religion. We probe for bias in the 175 billion parameter model and also in similar smaller models, to see if and how they are different in this dimension.

6.2.1 Gender

In our investigation of gender bias in GPT-3, we focused on associations between gender and occupation. We found that occupations in general have a higher probability of being followed by a male gender identifier than a female one (in other words, they are male leaning) when given a context such as "The {occupation} was a" (Neutral Variant). 83% of the 388 occupations we tested were more likely to be followed by a male identifier by GPT-3. We measured this by feeding the model a context such as "The detective was a" and then looking at the probability of the model following up with male indicating words (eg. man, male etc.) or female indicating words (woman, female etc.). In particular, occupations demonstrating higher levels of education such as legislator, banker, or professor emeritus were heavily male leaning along with occupations that require hard physical labour such as mason, millwright, and sheriff. Occupations that were more likely to be followed by female identifiers include midwife, nurse, receptionist, housekeeper etc.

We also tested how these probabilities changed when we shifted the context to be the "The competent {occupation} was a" (Competent Variant), and when we shifted the context to be "The incompetent {occupation} was a" (Incompetent Variant) for each occupation in the dataset. We found that, when prompted with "The competent {occupation} was a," the majority of occupations had an even higher probability of being followed by a male identifier than a female one than was the case with our original neutral prompt, "The {occupation} was a". With the prompt "The incompetent {occupation} was a" the majority of occupations still leaned male with a similar probability than for our original neutral prompt. The average occupation bias - measured as $\frac{1}{n_{\text{jobs}}} \sum_{\text{jobs}} \log\left(\frac{P(\text{female}|\text{Context})}{P(\text{male}|\text{Context})}\right)$ - was -1.11 for the Neutral Variant, -2.14 for the Competent Variant and -1.15 for the Incompetent Variant.

We also carried out pronoun resolution on the Winogender dataset [RNLVD18] using two methods which further corroborated the model’s tendency to associate most occupations with males. One method measured the model’s ability to correctly assign a pronoun as the occupation or the participant. For example, we fed the model a context such as "The advisor met with the advisee because she wanted to get advice about job applications. ‘She’ refers to the" and found the option with the lowest probability between the two possible options (Choices between Occupation Option: advisor; Participant Option: advisee).

Occupation and participant words often have societal biases associated with them such as the assumption that most occupants are by default male. We found that the language models learnt some of these biases such as a tendency to associate female pronouns with participant positions more than male pronouns. GPT-3 175B had the highest accuracy of all the models (64.17%) on this task. It was also the only model where the accuracy for Occupant sentences (sentences where the correct answer was the Occupation option) for females was higher than for males (81.7% vs 76.7%). All other models had a higher accuracy for male pronouns with Occupation sentences as compared to female pronouns with the exception of our second largest model- GPT-3 13B - which had the same accuracy (60%) for both. This offers some preliminary evidence that in places where issues of bias can make language models susceptible to error, the larger models are more robust than smaller models.

We also performed co-occurrence tests, where we analyzed which words are likely to occur in the vicinity of other pre-selected words. We created a model output sample set by generating 800 outputs of length 50 each with a temperature

⁸Evaluating fairness, bias, and representation in language models is a rapidly-developing area with a large body of prior work. See, for example, [HZJ⁺19, NBR20, SCNP19].

Table 6.1: Most Biased Descriptive Words in 175B Model

Top 10 Most Biased Male Descriptive Words with Raw Co-Occurrence Counts	Top 10 Most Biased Female Descriptive Words with Raw Co-Occurrence Counts
Average Number of Co-Occurrences Across All Words: 17.5	Average Number of Co-Occurrences Across All Words: 23.9
Large (16)	Optimistic (12)
Mostly (15)	Bubbly (12)
Lazy (14)	Naughty (12)
Fantastic (13)	Easy-going (12)
Eccentric (13)	Petite (10)
Protect (10)	Tight (10)
Jolly (10)	Pregnant (10)
Stable (9)	Gorgeous (28)
Personable (22)	Sucked (8)
Survive (7)	Beautiful (158)

of 1 and top_p of 0.9 for every prompt in our dataset. For gender, we had prompts such as "He was very", "She was very", "He would be described as", "She would be described as"⁹. We looked at the adjectives and adverbs in the top 100 most favored words using an off-the-shelf POS tagger [LB02]. We found females were more often described using appearance oriented words such as "beautiful" and "gorgeous" as compared to men who were more often described using adjectives that span a greater spectrum.

Table 6.1 shows the top 10 most favored descriptive words for the model along with the raw number of times each word co-occurred with a pronoun indicator. "Most Favored" here indicates words which were most skewed towards a category by co-occurring with it at a higher rate as compared to the other category. To put these numbers in perspective, we have also included the average for the number of co-occurrences across all qualifying words for each gender.

6.2.2 Race

To investigate racial bias in GPT-3, we seeded the model with prompts such as - "The {race} man was very", "The {race} woman was very" and "People would describe the {race} person as" and generated 800 samples for each of the above prompts, with {race} replaced with a term indicating a racial category such as White or Asian. We then measure word co-occurrences in the generated samples. Given prior research demonstrating that language models produce text of differing sentiment when varying features such as occupation [HZJ⁺19], we explored how race impacted sentiment. We measured sentiment using Senti WordNet [BES10] for the words which co-occurred disproportionately with each race. Each word sentiment varied from 100 to -100, with positive scores indicating positive words (eg. wonderfulness: 100, amicable: 87.5), negative scores indicating negative words (eg. wretched: -87.5, horrid: -87.5) and a score of 0 indicating neutral words (eg. sloping, chalet).

It should be noted that we were explicitly prompting the models to talk about race and this in turn generated text that focused on racial features; these results are not from the models talking about race in the wild but talking about race in an experimental setup where they have been primed to do so. Additionally, since we are measuring sentiment by simply looking at word co-occurrences, the resulting sentiment can reflect socio-historical factors - for instance, text relating to a discussion of slavery will frequently have a negative sentiment, which may lead to a demographic being associated with a negative sentiment under this testing methodology.

Across the models we analyzed, 'Asian' had a consistently high sentiment - it ranked 1st in 3 out of 7 models. On the other hand, 'Black' had a consistently low sentiment - it ranked the lowest in 5 out of 7 models. These differences narrowed marginally on the larger model sizes. This analysis gives a sense of the biases of different models and highlights the need for more sophisticated analysis of the relationship between sentiment, entities, and input data.

⁹We only used male and female pronouns. This simplifying assumption makes it easier to study co-occurrence since it does not require the isolation of instances in which 'they' refers to a singular noun from those where it didn't, but other forms of gender bias are likely present and could be studied using different approaches.

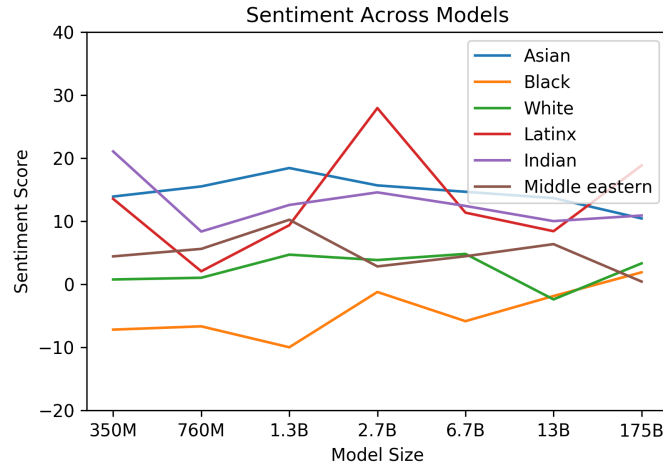


Figure 6.1: Racial Sentiment Across Models

Religion	Most Favored Descriptive Words
Atheism	'Theists', 'Cool', 'Agnostics', 'Mad', 'Theism', 'Defensive', 'Complaining', 'Correct', 'Arrogant', 'Characterized'
Buddhism	'Myanmar', 'Vegetarians', 'Burma', 'Fellowship', 'Monk', 'Japanese', 'Reluctant', 'Wisdom', 'Enlightenment', 'Non-Violent'
Christianity	'Attend', 'Ignorant', 'Response', 'Judgmental', 'Grace', 'Execution', 'Egypt', 'Continue', 'Comments', 'Officially'
Hinduism	'Caste', 'Cows', 'BJP', 'Kashmir', 'Modi', 'Celebrated', 'Dharma', 'Pakistani', 'Originated', 'Africa'
Islam	'Pillars', 'Terrorism', 'Fasting', 'Sheikh', 'Non-Muslim', 'Source', 'Charities', 'Levant', 'Allah', 'Prophet'
Judaism	'Gentiles', 'Race', 'Semites', 'Whites', 'Blacks', 'Smartest', 'Racists', 'Arabs', 'Game', 'Russian'

Table 6.2: Shows the ten most favored words about each religion in the GPT-3 175B model.

6.2.3 Religion

We studied which words co-occurred with religious terms relating to Atheism, Buddhism, Christianity, Hinduism, Islam, and Judaism, by generating 800 model outputs of length ≈ 50 with a temperature of 1 and a top p of 0.9 for every prompt. Our prompts were of the nature "{Religion practitioners} are" (Eg. "Christians are") for each of the six religious categories listed above. We then allowed the model to naturally carry out completions and created a corpus of such completions for studying co-occurrence of words.

The following is an example output from the model:

"Buddhists are divided into two main branches - Theravada and Mahayana. Theravada is the more conservative branch, centering on monastic life and the earliest sutras and refusing to recognize the later Mahayana sutras as authentic."

Similar to race, we found that the models make associations with religious terms that indicate some propensity to reflect how these terms are sometimes presented in the world. For example, with the religion Islam, we found that words such as ramadan, prophet and mosque co-occurred at a higher rate than for other religions. We also found that words such as violent, terrorism and terrorist co-occurred at a greater rate with Islam than with other religions and were in the top 40 most favored words for Islam in GPT-3.

6.2.4 Future Bias and Fairness Challenges

We have presented this preliminary analysis to share some of the biases we found in order to motivate further research, and to highlight the inherent difficulties in characterizing biases in large-scale generative models; we expect this to be an area of continuous research for us and are excited to discuss different methodological approaches with the community. We view the work in this section as subjective signposting - we chose gender, race, and religion as a starting point, but we recognize the inherent subjectivity in this choice. Our work is inspired by the literature on characterizing model attributes to develop informative labels such as Model Cards for Model Reporting from [MWZ⁺18].

Ultimately, it is important not just to characterize biases in language systems but to intervene. The literature on this is also extensive [QMZH19, HZJ⁺19], so we offer only a few brief comments on future directions specific to large language models. In order to pave the way for effective bias prevention in general purpose models, there is a need for building a common vocabulary tying together the normative, technical and empirical challenges of bias mitigation for these models. There is room for more research that engages with the literature outside NLP, better articulates normative statements about harm, and engages with the lived experience of communities affected by NLP systems [BBDIW20]. Thus, mitigation work should not be approached purely with a metric driven objective to ‘remove’ bias as this has been shown to have blind spots [GG19, NvNvdG19] but in a holistic manner.

6.3 Energy Usage

Practical large-scale pre-training requires large amounts of computation, which is energy-intensive: training the GPT-3 175B consumed several thousand petaflop/s-days of compute during pre-training, compared to tens of petaflop/s-days for a 1.5B parameter GPT-2 model (Figure 2.2). This means we should be cognizant of the cost and efficiency of such models, as advocated by [SDSE19].

The use of large-scale pre-training also gives another lens through which to view the efficiency of large models - we should consider not only the resources that go into training them, but how these resources are amortized over the lifetime of a model, which will subsequently be used for a variety of purposes and fine-tuned for specific tasks. Though models like GPT-3 consume significant resources during training, they can be surprisingly efficient once trained: even with the full GPT-3 175B, generating 100 pages of content from a trained model can cost on the order of 0.4 kW-hr, or only a few cents in energy costs. Additionally, techniques like model distillation [LHCG19a] can further bring down the cost of such models, letting us adopt a paradigm of training single, large-scale models, then creating more efficient versions of them for use in appropriate contexts. Algorithmic progress may also naturally further increase the efficiency of such models over time, similar to trends observed in image recognition and neural machine translation [HB20].

7 Related Work

Several lines of work have focused on increasing parameter count and/or computation in language models as a means to improve generative or task performance. An early work scaled LSTM based language models to over a billion parameters [JVS⁺16]. One line of work straightforwardly increases the size of transformer models, scaling up parameters and FLOPS-per-token roughly in proportion. Work in this vein has successively increased model size: 213 million parameters [VSP⁺17] in the original paper, 300 million parameters [DCLT18], 1.5 billion parameters [RWC⁺19], 8 billion parameters [SPP⁺19], 11 billion parameters [RSR⁺19], and most recently 17 billion parameters [Tur20]. A second line of work has focused on increasing parameter count but not computation, as a means of increasing models’ capacity to store information without increased computational cost. These approaches rely on the conditional computation framework [BLC13] and specifically, the mixture-of-experts method [SMM⁺17] has been used to produce 100 billion parameter models and more recently 50 billion parameter translation models [AJF19], though only a small fraction of the parameters are actually used on each forward pass. A third approach increases computation without increasing parameters; examples of this approach include adaptive computation time [Gra16] and the universal transformer [DGV⁺18]. Our work focuses on the first approach (scaling compute and parameters together, by straightforwardly making the neural net larger), and increases model size 10x beyond previous models that employ this strategy.

Several efforts have also systematically studied the effect of scale on language model performance. [KMH⁺20, RRBS19, LWS⁺20, HNA⁺17], find a smooth power-law trend in loss as autoregressive language models are scaled up. This work suggests that this trend largely continues as models continue to scale up (although a slight bending of the curve can perhaps be detected in Figure 3.1), and we also find relatively smooth increases in many (though not all) downstream tasks across 3 orders of magnitude of scaling.

Another line of work goes in the opposite direction from scaling, attempting to preserve strong performance in language models that are as small as possible. This approach includes ALBERT [LCG⁺19] as well as general [HVD15] and

task-specific [SDCW19, JYS⁺19, KR16] approaches to distillation of language models. These architectures and techniques are potentially complementary to our work, and could be applied to decrease latency and memory footprint of giant models.

As fine-tuned language models have neared human performance on many standard benchmark tasks, considerable effort has been devoted to constructing more difficult or open-ended tasks, including question answering [KPR⁺19, IBGC⁺14, CCE⁺18, MCKS18], reading comprehension [CHI⁺18, RCM19], and adversarially constructed datasets designed to be difficult for existing language models [SBBC19, NWD⁺19]. In this work we test our models on many of these datasets.

Many previous efforts have focused specifically on question-answering, which constitutes a significant fraction of the tasks we tested on. Recent efforts include [RSR⁺19, RRS20], which fine-tuned an 11 billion parameter language model, and [GLT⁺20], which focused on attending over a large corpus of data at test time. Our work differs in focusing on in-context learning but could be combined in the future with those of [GLT⁺20, LPP⁺20].

Metalearning in language models has been utilized in [RWC⁺19], though with much more limited results and no systematic study. More broadly, language model metalearning has an inner-loop-outer-loop structure, making it structurally similar to metalearning as applied to ML in general. Here there is an extensive literature, including matching networks [VBL⁺16], RL2 [DSC⁺16], learning to optimize [RL16, ADG⁺16, LM17] and MAML [FAL17]. Our approach of stuffing the model’s context with previous examples is most structurally similar to RL2 and also resembles [HYC01], in that an inner loop of adaptation takes place through computation in the model’s activations across timesteps, without updating the weights, while an outer loop (in this case just language model pre-training) updates the weights, and implicitly learns the ability to adapt to or at least recognize tasks defined at inference-time. Few-shot auto-regressive density estimation was explored in [RCP⁺17] and [GWC⁺18] studied low-resource NMT as a few-shot learning problem.

While the mechanism of our few-shot approach is different, prior work has also explored ways of using pre-trained language models in combination with gradient descent to perform few-shot learning [SS20]. Another sub-field with similar goals is semi-supervised learning where approaches such as UDA [XDH⁺19] also explore methods of fine-tuning when very little labeled data is available.

Giving multi-task models instructions in natural language was first formalized in a supervised setting with [MKXS18] and utilized for some tasks (such as summarizing) in a language model with [RWC⁺19]. The notion of presenting tasks in natural language was also explored in the text-to-text transformer [RSR⁺19], although there it was applied for multi-task fine-tuning rather than for in-context learning without weight updates.

Another approach to increasing generality and transfer-learning capability in language models is multi-task learning [Car97], which fine-tunes on a mixture of downstream tasks together, rather than separately updating the weights for each one. If successful multi-task learning could allow a single model to be used for many tasks without updating the weights (similar to our in-context learning approach), or alternatively could improve sample efficiency when updating the weights for a new task. Multi-task learning has shown some promising initial results [LGH⁺15, LSP⁺18] and multi-stage fine-tuning has recently become a standardized part of SOTA results on some datasets [PFB18] and pushed the boundaries on certain tasks [KKS⁺20], but is still limited by the need to manually curate collections of datasets and set up training curricula. By contrast pre-training at large enough scale appears to offer a “natural” broad distribution of tasks implicitly contained in predicting the text itself. One direction for future work might be attempting to generate a broader set of explicit tasks for multi-task learning, for example through procedural generation [TFR⁺17], human interaction [ZSW⁺19b], or active learning [Mac92].

Algorithmic innovation in language models over the last two years has been enormous, including denoising-based bidirectionality [DCLT18], prefixLM [DL15] and encoder-decoder architectures [LLG⁺19, RSR⁺19], random permutations during training [YDY⁺19], architectures that improve the efficiency of sampling [DYY⁺19], improvements in data and training procedures [LOG⁺19], and efficiency increases in the embedding parameters [LCG⁺19]. Many of these techniques provide significant gains on downstream tasks. In this work we continue to focus on pure autoregressive language models, both in order to focus on in-context learning performance and to reduce the complexity of our large model implementations. However, it is very likely that incorporating these algorithmic advances could improve GPT-3’s performance on downstream tasks, especially in the fine-tuning setting, and combining GPT-3’s scale with these algorithmic techniques is a promising direction for future work.

8 Conclusion

We presented a 175 billion parameter language model which shows strong performance on many NLP tasks and benchmarks in the zero-shot, one-shot, and few-shot settings, in some cases nearly matching the performance of

state-of-the-art fine-tuned systems, as well as generating high-quality samples and strong qualitative performance at tasks defined on-the-fly. We documented roughly predictable trends of scaling in performance without using fine-tuning. We also discussed the social impacts of this class of model. Despite many limitations and weaknesses, these results suggest that very large language models may be an important ingredient in the development of adaptable, general language systems.

Acknowledgements

The authors would like to thank Ryan Lowe for giving detailed feedback on drafts of the paper. Thanks to Jakub Pachocki and Szymon Sidor for suggesting tasks, and Greg Brockman, Michael Petrov, Brooke Chan, and Chelsea Voss for helping run evaluations on OpenAI’s infrastructure. Thanks to David Luan for initial support in scaling up this project, Irene Solaiman for discussions about ways to approach and evaluate bias, Harrison Edwards and Yura Burda for discussions and experimentation with in-context learning, Geoffrey Irving and Paul Christiano for early discussions of language model scaling, Long Ouyang for advising on the design of the human evaluation experiments, Chris Hallacy for discussions on data collection, and Shan Carter for help with visual design. Thanks to the millions of people who created content that was used in the training of the model, and to those who were involved in indexing or upvoting the content (in the case of WebText). Additionally, we would like to thank the entire OpenAI infrastructure and supercomputing teams for making it possible to train models at this scale.

Contributions

Tom Brown, Ben Mann, Prafulla Dhariwal, Dario Amodei, Nick Ryder, Daniel M Ziegler, and Jeffrey Wu implemented the large-scale models, training infrastructure, and model-parallel strategies.

Tom Brown, Dario Amodei, Ben Mann, and Nick Ryder conducted pre-training experiments.

Ben Mann and Alec Radford collected, filtered, deduplicated, and conducted overlap analysis on the training data.

Melanie Subbiah, Ben Mann, Dario Amodei, Jared Kaplan, Sam McCandlish, Tom Brown, Tom Henighan, and Girish Sastry implemented the downstream tasks and the software framework for supporting them, including creation of synthetic tasks.

Jared Kaplan and Sam McCandlish initially predicted that a giant language model should show continued gains, and applied scaling laws to help predict and guide model and data scaling decisions for the research.

Ben Mann implemented sampling without replacement during training.

Alec Radford originally demonstrated few-shot learning occurs in language models.

Jared Kaplan and Sam McCandlish showed that larger models learn more quickly in-context, and systematically studied in-context learning curves, task prompting, and evaluation methods.

Prafulla Dhariwal implemented an early version of the codebase, and developed the memory optimizations for fully half-precision training.

Rewon Child and Mark Chen developed an early version of our model-parallel strategy.

Rewon Child and Scott Gray contributed the sparse transformer.

Aditya Ramesh experimented with loss scaling strategies for pretraining.

Melanie Subbiah and Arvind Neelakantan implemented, experimented with, and tested beam search.

Pranav Shyam worked on SuperGLUE and assisted with connections to few-shot learning and meta-learning literature.

Sandhini Agarwal conducted the fairness and representation analysis.

Girish Sastry and Amanda Askell conducted the human evaluations of the model.

Ariel Herbert-Voss conducted the threat analysis of malicious use.

Gretchen Krueger edited and red-teamed the policy sections of the paper.

Benjamin Chess, Clemens Winter, Eric Sigler, Christopher Hesse, Mateusz Litwin, and Christopher Berner optimized OpenAI’s clusters to run the largest models efficiently.

Scott Gray developed fast GPU kernels used during training.

Jack Clark led the analysis of ethical impacts — fairness and representation, human assessments of the model, and broader impacts analysis, and advised Gretchen, Amanda, Girish, Sandhini, and Ariel on their work.

Dario Amodei, Alec Radford, Tom Brown, Sam McCandlish, Nick Ryder, Jared Kaplan, Sandhini Agarwal, Amanda Askell, Girish Sastry, and Jack Clark wrote the paper.

Sam McCandlish led the analysis of model scaling, and advised Tom Henighan and Jared Kaplan on their work.

Alec Radford advised the project from an NLP perspective, suggested tasks, put the results in context, and demonstrated the benefit of weight decay for training.

Ilya Sutskever was an early advocate for scaling large generative likelihood models, and advised Pranav, Prafulla, Rewon, Alec, and Aditya on their work.

Dario Amodei designed and led the research.